

Prior-Based Label Differential Privacy via Secure Two-Party Computation

Amit Agarwal*
UIUC
Champaign, IL, USA
amita2@illinois.edu

Stanislav Peceny*
Georgia Tech
Atlanta, GA, USA
stan.peceny@gatech.edu

Mariana Raykova
Google
New York, NY, USA
marianar@google.com

Phillipp Schoppmann
Google
New York, NY, USA
schoppmann@google.com

Karn Seth
Google
New York, NY, USA
karn@google.com

Abstract

Differential privacy (DP) is a fundamental technique used in machine learning (ML) training for protecting the privacy of sensitive individual user data. In the past few years, a new approach for combining *prior-based* Local Differential Privacy (LDP) mechanisms with a relaxed DP criterion, known as Label DP, has shown great promise in increasing the utility of the final trained model without compromising on the DP privacy budget. In this work, we identify a crucial privacy gap in the current implementations of these prior-based LDP mechanisms, namely the leakage of sensitive priors. We address the challenge of implementing such LDP mechanisms *without* leaking any information about the priors while preserving the efficiency and accuracy of the current insecure implementations. To that end, we design simple and efficient secure two-party computation (2PC) protocols for addressing this challenge, implement them, and perform end-to-end testing on standard datasets such as MNIST, CIFAR-10. Our empirical results indicate that the added security benefit essentially comes almost for “free” in the sense that the gap between the current insecure implementations and our proposed secure version, in terms of runtime overhead and accuracy degradation, is minimal. E.g., for CIFAR-10, with strong DP privacy parameter, the additional runtime due to 2PC is $\approx 3.9\%$ over WAN with 0.4% decrease in accuracy over an insecure (non-2PC) approach.

Keywords

Privacy-Preserving Deep Learning, Label Differential Privacy

1 Introduction

Training machine learning (ML) models on user population data is a core capability leveraged in applications that aim to understand user behavior and preferences to be able to make meaningful predictions about them [2, 8, 12, 13, 37, 38]. The challenge in these scenarios lies in the sensitivity of the input data for training, which belongs to many users. Differential Privacy (DP) [30] is a privacy definition, which formalizes the idea that the output of a computation that processes a dataset should not leak information about individual records in that dataset. It has often been used as a privacy measure for the output from ML training on private user data.

There are generally two avenues towards achieving DP for ML training. One of them relies on the notion of local Differential Privacy (LDP), where each user applies a DP mechanism to its individual contribution and the rest of the training is done as a postprocessing on a differentially private data release. The second approach relies on centralized mechanisms that need as input the original users’ data and the computation process itself adds the appropriate randomization to achieve DP output. While LDP mechanisms present a clean privacy argument, they achieve less utility for the same privacy budget.

In the case of the centralized mechanisms, utility is maximized, but one needs to address the challenges on how to protect the input data while the final DP output is computed. Cryptographic techniques for secure computation (two-party 2PC or multi-party MPC) [9, 39, 72] provide a solution for this question and they have been applied in different contexts with different architectures. In the setting of a single server and many clients, federated learning techniques [8, 12, 13], which leverage the MPC functionality for secure aggregation [13], have been a popular solution. Different approaches rely on several non-colluding servers to execute secure computation protocols for training on the secret-shared data of the clients [4, 44, 62, 67–69]. The latter removes the need for interaction with the users from single-server solutions but adds the non-collusion assumption. Moreover, the efficiency of secure computation for general ML training with even a small number of parties remains a challenge.

Label DP. A recent line of work [17, 34, 37, 38, 58, 65] considers a new notion of Differential Privacy called *label DP*, where the training features are assumed to be known in the clear by the training party (server), and the goal is to only protect the labels (held by clients). A label DP trained model provides privacy only with respect to the party with the features and it needs to be the only recipient of the output model. This applies to scenarios where a service provider is trying to compute a model over its user data. It has first-party context with its users when the features are generated, but the labels are generated in a different third-party context. This is the case in many application scenarios: in user cross-site activities such as online advertising, where features become available at impression time (when a user views an ad) and labels become available during conversion (e.g., when a user buys the advertised product); in recommendation systems where user

*Part of this work was done while the author was an intern at Google (New York, NY).

preferences choices are available to the service providers, but ratings need to be protected; and in medical analyses where patient demographics and general information are available while the test for a particular disease is sensitive. The Private Advertising Technology Working Group (PATWG) at W3C has set forward to solve the question for machine learning training in the setting where the features are known to one party who will receive the model, and thus only the labels need to be protected [61]. We emphasize that in all the aforementioned application scenarios, Label DP is *not* a weaker privacy notion compared to standard DP.

The local Differential Privacy (LDP) approach can be applied to the setting of label DP by having all users use the Randomized Response (RR) mechanism [70] to release their labels to the party who knows the features and can then train a label DP model. However, this still suffers from the utility disadvantages of LDP. Ghazi et al. [37, 38] introduced new techniques for Randomized Response that leverage the prior knowledge about the data, e.g., the distribution of label values. In particular, this solution changes the distribution from which random labels are sampled when the real values are not reported, to be consistent with the prior rather than be uniformly random. I.e., the mechanism is biased by the prior. Ghazi et al. [37] show that the accuracy gain of models trained using prior-based Randomized Response over non-prior-based Randomized Response can be as high as $\approx 8\%$ for moderate DP privacy parameter regimes and tends to increase even further when higher DP is desired.

Need for protecting the privacy of prior. The prior-dependent Randomized Response techniques have been adopted in recent papers [37, 38] to improve the quality of the trained model for classification and regression tasks. These works compute the priors used for the biased Randomized Response algorithm executed by the clients in different ways. Some of the techniques precompute the priors directly on the input label data with aggregate computation, e.g., histograms estimation. Other techniques update the prior throughout the training using both the feature and the label information. Priors that are computed on the sensitive user data, need to provide DP properties.

These existing solutions enforce the training party (holding the features and prior) to leak the sensitive priors to the users/clients in order to enable them to compute their contributions using the biased Randomized Response algorithm. In most application scenarios, the user population is an open set where any party can pose as a user in the system and participate in the model training protocol. In fact, adversaries can often control¹ a fraction of the users in the system. From this perspective, users should not be trusted to receive any sensitive information about other users as well as the party computing the model. And in the solution using classical local DP mechanisms, the users indeed do not receive any information. While the label DP property of the priors provides a level of protection for the labels of individual users, the approach of giving the priors to the clients effectively increases the information leaked to any arbitrary party compared with pure Local DP setting. This holds even if the features are public because the priors can be an arbitrary

function of (DP randomized) labels of *all* the clients. While clients may be comfortable revealing a randomized version of their label to a server, they may be less comfortable if data derived from that label, such as the prior, is shared with all other clients.

Furthermore, while the label DP priors protect individual labels, they do not protect user population level information, which could potentially be sensitive information for the model owner in the case of proprietary models. For example, the prior could be statistics about the users of a certain service, which the service owner may not want to reveal to its competitors. There are also settings where the features are not public information, but are information that the party building the model knows about the users of its service. In these cases, priors that are computed using the features as well, do not guarantee Differential Privacy for the individual user features. We note that the post-processing property of Label DP only guarantees privacy for the labels but not the features.

As a concrete example, the model owner could be an ad tech organization holding the features of a proprietary ads dataset (akin to Criteo dataset [28]) about users that derives prior based on this proprietary information. In this case, the priors amount to the model’s prediction on that client’s features. Since the model is only label-DP, it may not protect the other features used during training. Thus, revealing this prior to a specific user may leak information about *other* users’ features, which are sensitive. Furthermore, for a sensitive model, releasing such prior information to the clients can also leak information about the proprietary dataset and model to competitors, potentially allowing them to gain advantage. This is known as “Model extraction attack” [40, 56, 66], where priors may reveal proprietary model parameters or decision boundaries, enabling competitors to reverse engineer the model.

1.1 Our Contributions and Techniques

In this work, we show that revealing the priors to the users is *not* necessary to use the label DP solutions that rely on prior-based Randomized Response mechanisms. We present a novel solution that leverages secure two-party computation (2PC) between the client and the server to train the ML model. This enables the server to obtain the client’s label value, computed with Randomized Response with prior [37] mechanism, without the client learning any information about the prior and without the server learning anything more about the real label of the client than the output of the prior-based Randomized Response mechanism. This approach is illustrated in Figure 1.

At the core of our techniques is a novel protocol for joint sampling from a non-uniform distribution, where the distribution parameters are known by only one of the two parties (the server holding priors) and should remain secret to the other (the client). To that end, we take a two-fold approach. First, we algorithmically simplify the non-uniform sampling process in prior-based RR mechanisms by representing the underlying distribution as a composition of uniform sampling and a biased bit (Bernoulli) sampling. We do this because it seems non-trivial to construct an *efficient* secure protocol for directly emulating the non-uniform sampling procedure (and we are unaware of any prior work that provides such a protocol). Our representation results in a distribution that is identical to the original distribution while making the sampling

¹ We clarify that the term “control”, as typically used in MPC papers, can refer to either “semi-honest corruption” or “malicious corruption”. We only deal with “semi-honest corruption” in this paper and this model is described in detail in Section 3.2 and appendix A.

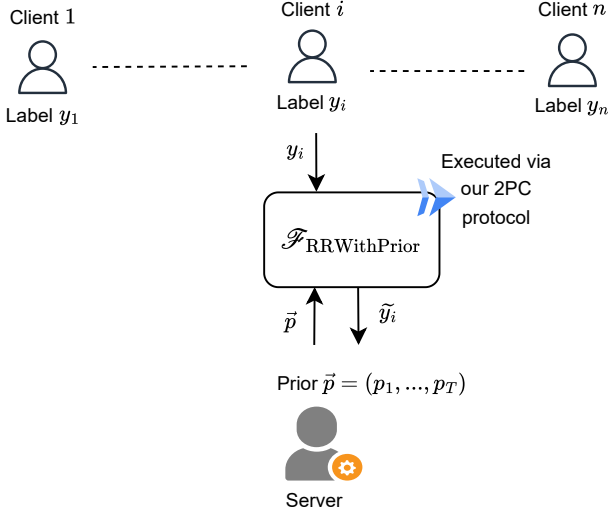


Figure 1: Securely randomizing the private label y_i (held by client i) using private prior $\vec{p}$ (held by the server) via an ideal functionality $\mathcal{F}_{RRWithPrior}$, for which we provide secure 2PC protocol. The functionality outputs the perturbed label $\tilde{y}_i$ to the server. After invoking the functionality between each client and the server, the server obtains a list $\{\tilde{y}_i\}_{i \in [n]}$ of perturbed labels, which can be used for higher-level applications such as model training.

procedure more amenable to secure computation. Second, we design modular and efficient sub-protocols to securely sample from these simpler distributions and compose them. All of our protocols make black-box use of Oblivious Transfer (OT) functionality, which is known to be necessary and sufficient for any non-trivial secure computation in the two-party setting [52], and admits a practically efficient realization using IKNP OT extension [47].

One of the main challenges in designing a secure protocol for sampling from these simpler distributions, even after algorithmic simplification, is that the parameters of the distribution are known only to one of the parties. One particularly interesting case is where one of the parties, i.e. the server, holds a private subset $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}$, where $\mathcal{Y}$ is the public label set and $T^* = |\mathcal{Y}_{T^*}|$. Both parties, i.e. the client and the server, want to securely sample a random element y from $\mathcal{Y}_{T^*}$ while keeping the sampled output y in secret-shared form and also protecting the input privacy of the set $\mathcal{Y}_{T^*}$ along with its size T^* . We show how to achieve this task efficiently using two sequential calls to OT, where the parties exchange their roles in the OT protocol.

We implement our prior-based RR protocol and use it to train standard ML datasets such as MNIST and CIFAR-10. Our protocol does not impose significant communication overhead or latency for the clients. E.g., for training, the CIFAR-10 dataset with the privacy budget $\epsilon = 1$ (our most expensive experiment), the runtime cost added due to 2PC over plaintext is ≈ 1.3 minutes, which amounts to $\approx 3.9\%$ overhead. This is a small fraction of the total 33.49min training cost. In total, we communicate 415MB and use 10 rounds. At the same time, our technique does not noticeably degrade the accuracy

of the resulting model. Specifically, we observe up to 0.7% difference, which can be reduced with higher fixed-point precision in exchange for a modest increase in the runtime and communication costs (or by using a better model). We also separately benchmark our prior-based RR protocol without the training procedure. To compute prior-based RR with 10 fractional bits and a label set of size 10, we require 1.55KB of communication and a runtime of 2.39ms on LAN and 321.25ms on WAN.

2 Related Work

Differentially private secure ML training. DP has been applied to ML training to achieve privacy for the final outputs. DP-SGD [2] presented a method to train differentially private models using stochastic gradient descent. However, this work assumes a trusted party that can execute the DP mechanism.

There are different approaches that avoid the need for a trusted party. E.g., they directly use local DP mechanisms such as Randomized Response [70] and other variants [3, 35, 37, 38]. Many of them require the clients to have additional knowledge such as priors. Another intermediate approach has been the shuffle DP model, which assumes there is a shuffler that shuffles all LDP contributions, before sending them to an analyzer who builds models [11, 20, 33].

A different technique is federated learning [8, 12, 13], where the training protocol is an interactive process between the server and the clients. In each iteration, the clients compute a gradient update with respect to the current model they receive. The server then obtains a differentially private aggregate of the gradient updates across clients. Federated learning solutions that avoid placing trust into the server use a secure aggregation protocol [13] that prevents the server from seeing anything but the aggregated gradient (where the clients themselves insert the DP noise).

There are also works that explore running the model training in secure computation across a small number of non-colluding parties [4, 44, 62, 67–69] (some works refer to this as vertical federated learning). A recent paper [73] aims to apply the label DP notion in the setting of distributed MPC training. Such solutions provide less efficiency since they need to execute large functionalities in MPC. Additionally, jointly sampling noise from the correct DP distribution is also a complex functionality and few papers do this in a fully distributed setting.

We note that none of these prior works address the question of secure evaluation of prior-based RR mechanism to enable label DP functionality. We are the first work to do so by addressing the unique challenges related to securely and efficiently noising the labels in this setting.

Secure noise generation for DP. Starting from the work of Dwork et al. [29], many followup works [10, 19, 25, 32, 50, 64, 71] explore the problem of securely sampling noise across two or more parties for various DP-relevant distributions. We note that in all of these works, the parameter of the noise distribution is *public*, while in our setting, the parameter is *private* for one of the parties. One exception is the work of Choi et al. [21], which studies noise sampling from private distributions. However, the focus of [21] is to construct protocols where the communication cost is sublinear in the description size of the distribution and their protocols require

Fully Homomorphic Encryption (FHE) to achieve this task. In contrast, all of our protocols require Oblivious Transfer (OT), which is computationally much more efficient than FHE.²

3 Preliminaries

3.1 Notation

In the context of a classification task, we will use $\mathcal{X}$ and $\mathcal{Y}$ to denote the set of possible features and labels, respectively. We will use T to denote the size of $\mathcal{Y}$ and assume without loss of generality that $\mathcal{Y} = \{0, \dots, T-1\}$. For a given scale (also called fixed-point precision) $f \in \mathbb{N}$, we use $x_{\text{fix}} \in [0, 2^f]$ to denote the fixed-point representation of a fractional real-valued $x \in [0, 1]$ where $x_{\text{fix}} = \lfloor x \cdot 2^f \rfloor$.

3.2 Secure Two-Party Computation (2PC)

Security model. In 2PC, each party has a private input and they are interested in computing a function on their joint inputs in a way that both parties only learn the final output without learning any extra information about the other party's input. Our protocol constructions, for different functions of interest, are in this two-party setting and provide *semi-honest security* [39] (also known as passive security). I.e., the parties are assumed to follow the prescribed protocol, but can glean extra information passively from the protocol transcript. We denote the two parties by P_0 and P_1 . The security in such a model is formally captured by the standard real/ideal paradigm whereby the view of an adversary in the real-world, which corrupts one of the parties (either P_0 or P_1), can be efficiently simulated in the ideal world where parties interact with a trusted third party (also called the ideal functionality). This third party takes the inputs of all parties and outputs the function evaluated on the joint set of all inputs. A detailed description of the security model is in Appendix A.

Rationale behind the security model. The aforementioned model fits well in our specific Label DP application where the two parties are the server and the client, with a prior and a label as their private inputs, respectively. The function of interest is a Randomized Response with prior mechanism. Moreover, in practical settings, the server (e.g. Amazon, Google, Meta, etc.) will be obligated by an authority (e.g. government) to follow the protocol instructions, but not collect sensitive client information. Our modeling choice is not merely theoretical; it has been deployed in practice e.g. for collecting user-statistics for browser telemetry [1] and measuring advertising conversions [68]. These systems assume one or more semi-honest servers to ensure client privacy.³

In a real-world setting involving multiple clients (each having a private label) and a single server (having a private prior), each client would execute a separate 2PC session with the server. In such a setting, our security model ensures that even if all the clients collude

²We do not focus on the goal of achieving sublinear communication as the description size of our distribution is practically small.

³In principle, one can also try to construct protocols in the malicious setting, where we do not assume the corrupt parties follow protocol instructions. Semi-honest model is commonly the first step to obtaining secure protocols in the malicious setting, which is an interesting open problem for future work. Also, in our application, where an authority can enforce parties to comply with the protocol instructions, it might be counter-productive to deploy a maliciously secure protocol as it will add unnecessary runtime and communication overhead.

and choose their labels arbitrarily, they will not learn any information about the priors (which is private information to the server). This follows by the Universal Composability (UC) guarantee [18] of the 2PC protocol as described in Appendix A, Remark A.1.

Offline-online 2PC. A 2PC protocol Π may sometimes be divided into an offline preprocessing phase Π_{offline} that is independent of parties' inputs, and an online phase Π_{online} that depends on parties' inputs [14, 48]. The security model still remains the same. This form of offline-online split is mainly aimed towards efficiency as the parties can perform the offline part anytime before the inputs arrive and then store the pre-processed information for future use in the online part. We separately benchmark the offline/online cost of our proposed protocols in addition to presenting the total cost.

Secret sharing. We use $\llbracket x \rrbracket^{\mathcal{R}}$ to denote an additive sharing of x in ring $\mathcal{R}$. We drop the superscript $\mathcal{R}$ when it is clear from context. Our default choice of $\mathcal{R}$ in this work is $\mathbb{Z}_T$ where T denotes the number of classification labels. We write $\llbracket x \rrbracket = (\llbracket x \rrbracket_0, \llbracket x \rrbracket_1)$ to denote that P_0 and P_1 get shares $\llbracket x \rrbracket_0$ and $\llbracket x \rrbracket_1$ respectively, such that $\llbracket x \rrbracket_0 + \llbracket x \rrbracket_1 = x$ in $\mathcal{R}$. An additive sharing is random if $\llbracket x \rrbracket_0$ and $\llbracket x \rrbracket_1$ are uniformly distributed in $\mathcal{R}$ subject to $\llbracket x \rrbracket_0 + \llbracket x \rrbracket_1 = x$. Analogously, we use $\langle b \rangle$ to denote a random XOR-sharing of a bit $b \in \{0, 1\}$, consisting of bits $\langle b \rangle_0$ and $\langle b \rangle_1$ such that $\langle b \rangle_0 \oplus \langle b \rangle_1 = b$.

Oblivious Transfer functionality. We use $\mathcal{F}_{\text{OT}}^{(N), \mathbb{G}}$ to denote an ideal 1-out-of- N chosen Oblivious Transfer (OT) functionality [16, 60] w.r.t. group $\mathbb{G}$. The sender's input to this functionality are N elements, $s_0, \dots, s_{N-1}$, where each $s_i \in \mathbb{G}$, whereas the receiver's input is a choice index $c \in [0, N-1]$. The receiver receives the element s_c from the functionality, whereas the sender has no output. Our usual choice of $\mathbb{G}$ in this work will be the additive group of $\mathbb{Z}_T$ or $\mathbb{Z}_2$. Efficient realizations of this functionality rely on the IKNP OT extension [47] paradigm which uses a small number of λ "base" OTs, based on public-key style primitives, to generate a large number of λ^k OTs, for any constant k , using only cheap symmetric-key style primitives. Using the optimized OT extension in [5], we can generate 1-out-of-2 *Random* OT (ROT) correlations (where the inputs of sender and receiver are uniformly random) using a single round at an amortized cost of λ bits of communication from the sender to the receiver and no communication in the other direction. One can non-interactively convert $\log N$ instances of 1-out-of-2 ROT correlations into a single instance 1-out-of- N ROT correlation using [60]. An instance of 1-out-of- N ROT correlation can then be used to perform 1-out-of- N chosen OT $\mathcal{F}_{\text{OT}}^{(N), \mathbb{G}}$ (where the inputs of sender and receiver are arbitrary) using just 2 rounds - first round from the receiver to sender having $\log N$ bits and second round from the sender to receiver having $N \log |\mathbb{G}|$ bits.⁴

3.3 (Standard) Differential Privacy (DP) and Label Differential Privacy (Label DP)

At a high-level, Differential Privacy (DP) [31] ensures that the output of a randomized algorithm is nearly indistinguishable whether or not any single individual's data is included in the input dataset.

⁴While it is theoretically possible to reduce the cost of $\mathcal{F}_{\text{OT}}^{(N), \mathbb{G}}$ to $o(N)$ using techniques based on single server Private Information Retrieval (PIR) [27, 55], the cost benefit only shows up for large values of N . In our experiments, the value of N will be small (on the order of $N \approx 10$) so we do not use heavy-weight PIR-based optimization.

Formally, an algorithm $\mathcal{A}$ satisfies ϵ -DP⁵ if for all neighboring datasets D and D' (which differ by at most one element) and for any subset of outputs S :

$$\Pr[\mathcal{A}(D) \in S] \leq e^\epsilon \Pr[\mathcal{A}(D') \in S]$$

Label Differential Privacy (Label DP) [37, 38] is a relaxation of the standard DP, tailored towards ML tasks, where the privacy guarantee is only required w.r.t dataset labels whereas the dataset features are considered non-sensitive. The formal definition of ϵ -label DP is exactly same as ϵ -DP defined above with a slight modification to the notion of neighbouring datasets: Datasets D and D' are neighbors if they differ by at most one *label*. We refer the readers to Appendix B for a more detailed definition.

3.4 Prior-Based Randomized Response

Randomized response (RR) [70] is a classical technique for obtaining DP in the local setting (aka LocalDP setting). For any $T \in \mathbb{N}$, let $\mathcal{Y} = \{0, \dots, T-1\}$. Let $\epsilon \in \mathbb{R}_{\geq 0}$ be a privacy parameter. Then a Randomized Response mechanism with parameters T, ϵ (denoted by $\text{RR}_{\epsilon, T}$) takes as input $y \in \mathcal{Y}$ and returns a random sample $\tilde{y}$ drawn from the following probability distribution.

$$\Pr[\tilde{y} = y'] = \begin{cases} \frac{e^\epsilon}{e^\epsilon + T - 1} & ; y' = y \\ \frac{1}{e^\epsilon + T - 1} & ; y' \in \mathcal{Y} \setminus \{y\} \end{cases}$$

It is well-known that RR satisfies ϵ -DP guarantee. Next, we will describe some better alternatives of this classical RR which take into account prior information.

3.4.1 Randomized Response with Prior. Ghazi et al. [37] recently proposed a variant of the Randomized Response (RR) mechanism, termed Randomized Response with Prior (RRWithPrior). In vanilla RR, the private input $y \in \mathcal{Y}$ is randomized by either retaining the same input y with some fixed probability q or replacing with a uniformly random input from $\mathcal{Y}$ with probability $1 - q$. In contrast, RRWithPrior, as the name suggests, uses a prior probability distribution $\vec{p} = (p_1, \dots, p_T)$ to randomize the private input y in a way that maximizes the signal-to-noise ratio while maintaining (ϵ, δ) -DP guarantee w.r.t. y . Note that there is no DP requirement w.r.t. the prior $\vec{p}$.

In Algorithm 1, we describe the randomizer that is proposed in [37]. At a high-level, the algorithm finds a reduced set $\mathcal{Y}_{T^*}$ consisting of T^* labels and then performs vanilla RR on $\mathcal{Y}_{T^*}$ when $y \in \mathcal{Y}_{T^*}$. If $y \notin \mathcal{Y}_{T^*}$, it returns a uniformly random label from $\mathcal{Y}_{T^*}$. Ghazi et al. [37] prove that this approach satisfies ϵ -DP guarantee and also maximizes the objective function capturing the signal-to-noise ratio.

THEOREM 3.1 ([37]). *For all $T \in \mathbb{N}, \vec{p} \in \mathbb{R}^T, \epsilon \in \mathbb{R}_{\geq 0}$, RRWithPrior (Algorithm 1) is ϵ -DP (Definition B.2).*

Ghazi et al. [37] also show that in the Label DP setting, training a model using labels perturbed via RRWithPrior algorithm achieves a much better accuracy compared to performing the same training using labels perturbed via vanilla RR mechanism for any fixed privacy budget ϵ .

⁵The parameter ϵ is typically known as the privacy parameter/budget. A lower value of ϵ implies higher privacy.

Algorithm 1: RRWithPrior

Parameters: Number of labels T , Label set $\mathcal{Y} = \{0, \dots, T-1\}$, privacy parameter $\epsilon \in \mathbb{R}_{\geq 0}$, prior probability vector $\vec{p} = (p_1, \dots, p_T)$ where each $p_i \in \mathbb{R}$.

Inputs: Label $y \in \mathcal{Y}$.

Output: A perturbed label $\tilde{y} \in \mathcal{Y}$.

- 1 Compute $T^* = \arg\max_{t \in [T]} \left(\frac{e^\epsilon}{e^\epsilon + T - 1} \cdot (\sum_{i \in \mathcal{Y}_t} p_i) \right)$, where $\mathcal{Y}_t$ is the set of t labels with maximum p_i values.
- 2 Let $\mathcal{Y}_{T^*}$ be the set of T^* labels with maximum p_i values.
- 3 **if** $y \in \mathcal{Y}_{T^*}$ **then**
- 4 Define a random variable Y_1 with values in $\mathcal{Y}$ and let $\mathcal{D}_1$ be the induced probability distribution s.t.
- 5

$$\Pr[Y_1 = y'] = \begin{cases} \frac{e^\epsilon}{e^\epsilon + T^* - 1} & ; y' = y \\ \frac{1}{e^\epsilon + T^* - 1} & ; y' \in \mathcal{Y}_{T^*} \setminus \{y\} \\ 0 & ; y' \in \mathcal{Y} \setminus \mathcal{Y}_{T^*} \end{cases}$$

- 6 Sample $\tilde{y} \leftarrow \mathcal{D}_1$
 - 7 **else**
 - 8 Define a random variable Y_2 with values in $\mathcal{Y}$ and let $\mathcal{D}_2$ be the induced probability distribution s.t.
 - 9 Sample $\tilde{y} \leftarrow \mathcal{D}_2$
 - 10 **return** $\tilde{y}$.
-

3.4.2 Prior computation and model training. Ghazi et al. [37, 38] discuss different possible ways in which the prior distribution on the label set can be obtained. In some applications, these can be obtained from domain-specific knowledge or from models trained on publicly available data. In other applications, it might be suitable to spend a portion of the privacy budget to obtain priors based on the actual private labels held by the clients via an ϵ -DP mechanism. Besides these application and domain-specific approaches, they also describe a general domain-agnostic training framework that bootstraps from a uniform prior, and progressively updates the priors via iterative training. We emphasize two important points: i) Regardless of the specific approach used for computing the prior, these priors are always treated as a private input of the server as described in the introduction, ii) The ϵ -DP guarantee of prior-based RR mechanisms holds for *all* prior values. In Algorithm 2, we describe this approach, known as Label Privacy Multi-Stage Training (LP-MST), which makes a black-box use of RRWithPrior as a subroutine. Note that Line 6 in Algorithm 2 is the part where priors are computed as prediction probabilities of the intermediate trained model. In our experiments, we focus on this general approach for computing the priors and performing model training. The authors show that if RRWithPrior satisfies ϵ -DP guarantee, then LP-MST will satisfy ϵ -label DP guarantee that is captured by Theorem 3.2.⁶

THEOREM 3.2 ([37]). *Let $\epsilon \in \mathbb{R}_{\geq 0}$. If RRWithPrior (Algorithm 1) is ϵ -DP (Definition B.2), then for all feature sets X , $T \in \mathbb{N}$, $I \in \mathbb{N}$ and*

⁶We remark that while we stick to LP-MST approach in this work for computing the priors in our experiments, the secure protocols that we propose are not limited to it and can work with any prior that the server has independently of how the prior is computed.

for all training algorithms $\mathcal{A}$, LP-MST (Algorithm 2) is ϵ -Label DP (Definition B.4).

At a high-level, this holds because each private label y_i is queried only once in Line 7 of Algorithm 2 (highlighted in blue) via the ϵ -DP mechanism RRWithPrior. We note that the ϵ -Label DP guarantee of Algorithm 2 is not specific to RRWithPrior and works for any Randomized Response mechanism as long as it satisfies ϵ -DP. For example, one could substitute RRWithPrior in Line 7 of Algorithm 2 with RROnBins to obtain a version of Algorithm 2 tailored towards regression tasks while still satisfying ϵ -Label DP guarantee.

Algorithm 2: Label Privacy Multi-Stage Training (LP-MST)

Parameters: Feature set $\mathcal{X}$, number of labels T , label set $\mathcal{Y} = \{0, \dots, T-1\}$, privacy parameter $\epsilon \in \mathbb{R}_{\geq 0}$, number of iterations I , training algorithm $\mathcal{A}$.

Inputs: Dataset $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ where $(x_j, y_j) \in \mathcal{X} \times \mathcal{Y}$ for all $j \in [n]$.

Output: A trained model M .

```

1 Arbitrarily partition  $S$  into  $S^{(1)}, \dots, S^{(I)}$ 
2 Let  $M^{(0)}$  be the trivial model that always assigns equal probability
  to each class.
3 for  $i = 1$  to  $I$  :
4   Let  $\tilde{S}^{(i)} = \emptyset$ 
5   for  $(x_j, y_j) \in S^{(i)}$  :
6     Let  $\tilde{p} = (p_1, \dots, p_T)$  be the probabilities predicted by  $M^{(i-1)}$ 
      on  $x_j$ 
7     Let  $\tilde{y}_j := \text{RRWithPrior}_{T, \mathcal{Y}, \tilde{p}}(y_j)$ 
8     Add  $(x_j, \tilde{y}_j)$  to  $\tilde{S}^{(i)}$ 
9   Let  $M^{(i)}$  be the model resulting from training on  $\tilde{S}^{(1)}, \dots, \tilde{S}^{(i)}$ 
    using  $\mathcal{A}$ 
10 return  $M^{(I)}$ .
```

4 Securely Computing RR with Prior

Note that classical RR, described in the beginning of Section 3.4, admits a straightforward secure protocol wherein the client locally perturbs its private label y using the $\text{RR}_{\epsilon, T}$ mechanism and sends the perturbed label $\tilde{y}$ to the server.⁷ However, the same protocol cannot be used to securely compute prior-based RR mechanisms. Specifically, let us focus on the RRWithPrior mechanism described in Section 3.4. In this case, there are *two* private inputs, namely the label y (privately held by the client) and the prior vector $\tilde{p}$ (privately held by the server). Therefore, the protocol needs to operate jointly on the private input held by both parties while ensuring that the client learns nothing about $\tilde{p}$ and the server learns nothing about y except $\tilde{y}$.

Problem formulation. We observe that securely training a neural network in a Label DP setting using LP-MST (Algorithm 2) boils down to securely computing the randomizer RRWithPrior (Algorithm 1). This is because once the private labels are randomly perturbed using RRWithPrior, the perturbed labels can be released

⁷This protocol is secure only in the semi-honest setting which is the focus of our work. In the malicious setting, we will need to additionally ensure that the client is adding the DP noise from the right distribution. This is a much harder problem that we leave as part of future work.

in clear to the party holding the features which it can then use for performing training using LP-MST. In this section, we will design an approach to securely execute the RRWithPrior randomizer in a client-server setting. We will denote the server as P_0 and the client as P_1 . The setting is as follows: P_0 privately holds a prior vector $\tilde{p} \in \mathbb{R}^T$ whereas P_1 privately holds a label $y \in \mathcal{Y}$.⁸ They would like to execute Algorithm 1 securely in 2PC so that P_0 learns the output $\tilde{y}$ (i.e. perturbed label) in the clear. Note that in practice, many clients may each independently run RRWithPrior with the server. The protocol is agnostic to how clients interact with the system; clients can join at any time and run the protocol.

In terms of security, we require the following guarantees:

- For a semi-honest P_0 , the view is simulatable given only the perturbed label $\tilde{y}$ and prior vector $\tilde{p}$.
- For a semi-honest P_1 , the view is simulatable given only the label y .

To capture the privacy requirements more formally, we need to define an ideal functionality which precisely captures the task RRWithPrior (Algorithm 1) that we wish to securely compute. Before doing that, we will simplify the non-uniform sampling step in RRWithPrior (Algorithm 1) so that it is amenable to secure computation without modifying the underlying distribution.

Simplifying the non-uniform sampling. In Algorithm 3, we describe an equivalent and simplified way of performing the non-uniform sampling that happens in RRWithPrior (Steps 3-10 in Algorithm 1). At a high-level, this simplification represents the non-uniform distribution induced on $\tilde{y}$ in Algorithm 1 as a composition of uniform distribution and a biased bit (Bernoulli) distribution. Looking ahead, this simplified sampling procedure will enable a simplified design of the 2PC protocol via modular subprotocols.

Algorithm 3: Simplified sampling in RRWithPrior

```

1 Sample  $b_1 \leftarrow \text{Bernoulli}(q)$ , where  $q = \frac{e^\epsilon - 1}{e^\epsilon + T^* - 1}$ .
2 Compute  $b_2 := 1\{y \in \mathcal{Y}_{T^*}\}$ 
3 Sample  $z \leftarrow \mathcal{Y}_{T^*}$ 
4 If  $b_1 \wedge b_2 = 1$ , set  $\tilde{y} := y$ . Else set  $\tilde{y} := z$ 
5 return  $\tilde{y}$ .
```

We now briefly explain why the Steps 1-5 of Algorithm 3 are equivalent to Steps 3-10 of Algorithm 1. Suppose $y \in \mathcal{Y}_{T^*}$. In this case, $b_2 = 1$ and the output $\tilde{y}$ is y with probability $q + (1 - q)\frac{1}{T^*} = \frac{e^\epsilon}{e^\epsilon + T^* - 1}$, an element of $\mathcal{Y}_{T^*} \setminus \{y\}$ each with probability $(1 - q)\frac{1}{T^*} = \frac{1}{e^\epsilon + T^* - 1}$, an element of $\mathcal{Y} \setminus \mathcal{Y}_{T^*}$ each with probability 0. This distribution matches the distribution $\mathcal{D}_1$ in Algorithm 1. Now let's consider the case where $y \notin \mathcal{Y}_{T^*}$. In this case, $b_2 = 0$ and the output $\tilde{y}$ is simply z which is a uniformly random element from $\mathcal{Y}_{T^*}$. Hence, this distribution matches the distribution $\mathcal{D}_2$ in Algorithm 1.

⁸We note that the protocols that we construct in this work can also be executed in a 2-server and no client setting where the labels are secret-shared between the two servers (instead of being privately held by one party as in the client-server setting). This can be done by a straight-forward modification of one of our sub-protocols $\Pi_{\text{membership}}$ while other protocols will remain unmodified.

Defining the ideal functionality. Now we define an ideal functionality $\mathcal{F}_{\text{RRwithPrior}}$ in Figure 2 which captures RRWithPrior (Algorithm 1) where we replace Steps 3-10 of Algorithm 1 with Steps 1-5 of Algorithm 3 for simplification. The security of a protocol $\Pi_{\text{RRwithPrior}}$ realizing $\mathcal{F}_{\text{RRwithPrior}}$ is captured via the standard simulation based security notion that is described in Appendix A.

Functionality: $\mathcal{F}_{\text{RRwithPrior}}$

Public Parameters: Number of classes T , label set $\mathcal{Y} = \{0, \dots, T-1\}$, privacy parameter $\epsilon \in \mathbb{R}_{\geq 0}$, fixed point precision $f \in \mathbb{N}$.

- (1) Get a prior vector $\vec{p} = \{p_i\}_{i \in [T]}$ as input from P_0 where $p_i \in \mathbb{R}$, $p_i \in [0, 1]$ and $\sum_{i \in [T]} p_i = 1$.
- (2) Get a label $y \in \mathcal{Y}$ as input from P_1 .
- (3) Compute $T^* = \arg\max_{t \in [T]} \left(\frac{e^\epsilon}{e^\epsilon + t - 1} \cdot (\sum_{i \in \mathcal{Y}_t} p_i) \right)$, where $\mathcal{Y}_t$ is the set of t labels with maximum p_i values.
- (4) Fix $\mathcal{Y}_{T^*}$ to be the set of T^* labels with maximum p_i values.
- (5) Set $q_{\text{fix}} = \left\lfloor \frac{e^\epsilon - 1}{e^\epsilon + T^* - 1} \cdot 2^f \right\rfloor$.
- (6) Sample $b_1 \leftarrow \text{Bernoulli}(q_{\text{fix}}/2^f)$.
- (7) Compute $b_2 := 1\{y \in \mathcal{Y}_{T^*}\}$.
- (8) Sample $z \leftarrow \mathcal{Y}_{T^*}$.
- (9) If $b_1 \wedge b_2 = 1$, set $\tilde{y} := y$. Else set $\tilde{y} := z$.
- (10) Send $\tilde{y}$ to P_0 .

Figure 2: Ideal functionality $\mathcal{F}_{\text{RRwithPrior}}$ for RRWithPrior

We remark two important differences between $\mathcal{F}_{\text{RRwithPrior}}$ (Figure 2) and RRWithPrior (Algorithm 1). First, $\mathcal{F}_{\text{RRwithPrior}}$ treats the prior vector $\vec{p}$ as an input of party P_0 instead of public parameter. This, as explained earlier, is necessary to capture the privacy requirements of the prior $\vec{p}$ w.r.t. P_0 . Second, $\mathcal{F}_{\text{RRwithPrior}}$ introduces an additional parameter f which captures the fixed point precision to be used for representing the bias value q . This is needed because the secure protocol $\Pi_{\text{RRwithPrior}}$ realizing this functionality will be limited to performing computations in a finite fixed-point domain. We note that this restriction to fixed-point domain does not lead to leakage-based attacks [59] and has minimal effect on the ϵ as explained in Section 5.

Protocol construction. We first note that Steps 3-5 in $\mathcal{F}_{\text{RRwithPrior}}$ can be computed locally by P_0 to derive the set $\mathcal{Y}_{T^*}$ and q_{fix} since it only depends on the prior vector $\vec{p}$. The main challenge lies in computing Steps 6 - 10. Note that securely computing these steps requires the P_0 's set $\mathcal{Y}_{T^*}$, along with its size T^* , to be kept private (in addition to the keeping P_1 's label y private).

To design $\Pi_{\text{RRwithPrior}}$, a protocol that securely realizes the functionality $\mathcal{F}_{\text{RRwithPrior}}$, we will take a modular approach and design subprotocols for individually computing each step, from Steps 6 - 10, securely. These subprotocols will take either private or secret-shared input and then output the result in a secret-shared form to both parties. Table 1 shows the mapping between the above steps and their corresponding subprotocol names. Since Step 10 has a trivial protocol where both parties do a public reconstruction of $\tilde{y}$ (from Step 9) towards P_0 , we do not explicitly introduce a protocol for it.

Steps in $\mathcal{F}_{\text{RRwithPrior}}$	Subprotocols
6	Π_{biasBit}
7	$\Pi_{\text{membership}}$
8	Π_{rand}
9	$\Pi_{\text{mux}}^{(f)}$

Table 1: Mapping between $\mathcal{F}_{\text{RRwithPrior}}$ steps and corresponding subprotocols implementing them. These sub-protocols would be combined to derive the final protocol $\Pi_{\text{RRwithPrior}}$.

4.1 Subprotocols

In this section, we describe the construction of subprotocols from Table 1. To formally prove the security of our subprotocols, we define functionalities $\mathcal{F}_*$ in Appendix D corresponding to each of the subprotocols Π_* where $*$ $\in$ {rand, membership, sample-bias-bit, mux}. We then show that Π_* securely realizes the functionality $\mathcal{F}_*$ w.r.t a semi-honest adversary corrupting one of the parties.

4.1.1 Secure sampling from a private set. In this protocol, we have a public set $\mathcal{Y}_T = \{0, \dots, T-1\}$ and P_0 has a private set $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}_T$ of size T^* (which is also private). Both parties would like to securely sample (an arithmetic sharing of) a uniformly random element $y \in \mathcal{Y}_{T^*}$.

Protocol. We describe a protocol Π_{rand} for realizing this task in Figure 3. This protocol proceeds in two stages. At a high-level, parties first generate a sharing of random value $v \in [0, T^* - 1]$ and then, in the second stage, simply retrieve the sharing of v^{th} element in P_0 's private set $\mathcal{Y}_{T^*}$. We now explain each stage in detail.

Protocol: Π_{rand}

Public input: Set $\mathcal{Y} = \{0, \dots, T-1\}$ and arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

Private input: P_0 holds $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}$ where $T^* = |\mathcal{Y}_{T^*}|$.

Output: $\llbracket r \rrbracket$ where $r \leftarrow \mathcal{Y}_{T^*}$.

- (1) P_1 samples $z \leftarrow \mathbb{Z}_T$ and sets $\llbracket v \rrbracket_1 := z$.
- (2) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$:
 - P_1 acts as the OT sender and sets the i^{th} string (0-indexed) as $s_i = z \pmod{T}$, where $s_i \leftarrow [0, i]$.
 - P_0 acts as the OT receiver using $T^* - 1$ as the choice index.
 - P_0 retrieves $s_{T^*-1} = z$, denoted as $\llbracket v \rrbracket_0$.
- (3) P_0 computes $\tilde{\mathcal{Y}}_{T^*} := \pi(\mathcal{Y}_{T^*})$, where π is a random permutation.
- (4) P_0 samples $z' \leftarrow \mathbb{Z}_T$.
- (5) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$:
 - P_0 acts as the OT sender and sets the i^{th} OT string (0-indexed) $u_i = d_i - z' \pmod{T}$, where $d_i := (\llbracket v \rrbracket_0 + i \pmod{T})^{\text{th}}$ label in $\tilde{\mathcal{Y}}_{T^*}$. If $\llbracket v \rrbracket_0 + i \pmod{T} \notin \{0, \dots, T^* - 1\}$, then set $u_i = 0$.
 - P_1 acts as the OT receiver with the choice index $\llbracket v \rrbracket_1$.
 - P_1 retrieves $u_{\llbracket v \rrbracket_1}$.
- (6) P_0 outputs z' .
- (7) P_1 outputs $u_{\llbracket v \rrbracket_1}$.

Figure 3: Protocol Π_{rand} for secure sampling from private set.

In the first stage, P_1 samples a random mask z and then both parties invoke $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$ where P_1 acts as the OT sender and sets the i^{th} sender string to be a fresh random value in $[0, 1]$ masked by z . P_0 acts as the OT receiver using $T^* - 1$ as the choice index and ends up retrieving a value $\llbracket v \rrbracket_0$. Interpreting P_1 's private mask z as a share $\llbracket v \rrbracket_1$, it is clear that $(\llbracket v \rrbracket_0, \llbracket v \rrbracket_1)$ form a random sharing of a random value $v \in [0, T^* - 1]$.

In the second stage, both parties again invoke $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$ but this time with the sender and receiver roles reversed. The aim of this OT is to retrieve a sharing of the v^{th} element from P_0 's private set $\mathcal{Y}_{T^*}$. This can be done by having P_0 , acting as the OT sender, set up the OT sender strings in a way so that whenever P_1 , acting as the OT receiver, uses $\llbracket v \rrbracket_1$ as its choice index, then P_1 retrieves a sharing of the v^{th} element from $\mathcal{Y}_{T^*}$. P_0 can set-up the OT strings in this desired fashion as it knows its own share of v , which is $\llbracket v \rrbracket_0$, and can simply iterate over all possible values of $\llbracket v \rrbracket_1$ (there are T such possible values which is equal to the number of OT strings).

The protocol as described so far is not actually secure. The reason is that the value v is in the view of P_1 as it was the one who sampled it. As a concrete attack, suppose that all the OT sender strings in the first OT happen to be a masking of 0 (which can happen with some non-negligible probability). Then, P_1 knows that irrespective of what choice index P_0 uses, both will end up with a sharing of $v = 0$ at the end of stage one. This knowledge of v implies that P_1 knows exactly which element of P_0 's private set $\mathcal{Y}_{T^*}$ will it retrieve (in a shared form) in the second stage. This is a leakage which we do not expect in an ideal setting (see Figure 11, Appendix D.2). To mitigate this leakage, P_0 locally shuffles the set $\mathcal{Y}_{T^*}$ before starting the second stage. At a high-level, this shuffling breaks the correlation between v and the index of element retrieved from $\mathcal{Y}_{T^*}$.

Security and Efficiency. To formally prove security of Π_{rand} , we define an ideal functionality $\mathcal{F}_{\text{rand}}$ (Figure 11, Appendix D.2) and show that Π_{rand} securely realizes $\mathcal{F}_{\text{rand}}$ in the $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$ hybrid model (Theorem D.2, Appendix D.2). In terms of efficiency, the protocol simply performs two calls to the OT functionality $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_T}$ which incurs 1 round and $2\lambda \log T$ bits of communication in the offline phase. In the online phase, we have 4 rounds (since the two OTs need to be executed sequentially) and $2(T + 1) \log T$ bits of communication (across both parties).⁹

4.1.2 Secure bit sampling with private bias. In this protocol, P_0 holds a private bias value q_{fix} represented in fixed-point format with a public precision f . Both parties would like to securely sample (a Boolean sharing of) a biased bit b from the distribution $\text{Bernoulli}(\frac{q_{\text{fix}}}{2^f})$.

Protocol. We describe a protocol for realizing this task in Figure 4.

The idea behind this protocol is simple: Both parties invoke $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ where P_0 acts as the OT sender and sets a random subset of $q_{\text{fix}}/2^f$ fraction of the sender strings to be 1 and the rest to be 0. Then, P_1 acts as the OT receiver and simply retrieves a random OT sender

⁹As a minor optimization, we can reduce the number of rounds in the online phase from 4 to 3 by parallelizing the second round message of first OT (which is sent by P_1 as it is acting as the OT sender) with the first round message of the second OT (which is again sent by P_1 as it is acting as the OT receiver).

string. This will result in P_1 retrieving a bit b from $\text{Bernoulli}(\frac{q_{\text{fix}}}{2^f})$. To have the output bit b in a secret-shared form across both parties, instead of being available in clear to P_1 , P_0 can simply mask all the OT sender strings using a random mask bit z and then interpreting z as its share of bit b whereas P_1 interprets the OT retrieved bit as its share of bit b .

Protocol: Π_{biasBit}

Public input: Fixed point precision $f \in \mathbb{N}$.

Private input: P_0 holds a value $q_{\text{fix}} \in [0, 2^f]$.

Output: $\langle b \rangle$ where $b \leftarrow \text{Bernoulli}(\frac{q_{\text{fix}}}{2^f})$.

(1) P_0 samples $z \leftarrow \{0, 1\}$.

(2) P_0 samples a set $S \subseteq [0, 2^f)$ s.t. $|S| = q_{\text{fix}}$.

(3) P_1 samples $r \leftarrow [0, 2^f)$.

(4) P_0 and P_1 invoke $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$:

- P_0 acts as the sender. For all $i \in [0, 2^f)$, P_0 sets i^{th} sender bit $s_i = 1 \oplus z$ if $i \in S$ and $0 \oplus z$ otherwise.

- P_1 acts as the receiver using r as the choice index.

- P_1 retrieves s_r .

(5) P_0 outputs z .

(6) P_1 outputs s_r .

Figure 4: Protocol Π_{biasBit} for secure bit sampling with private bias.

Security and Efficiency. To formally prove the security of Π_{biasBit} , we define an ideal functionality $\mathcal{F}_{\text{biasBit}}$ (Figure 12 in Appendix D.3) and then show that Π_{biasBit} securely realizes $\mathcal{F}_{\text{biasBit}}$ in the $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ hybrid model (Theorem D.3 in Appendix D.3).

In terms of efficiency, the protocol simply performs a single call to the OT functionality $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ which incurs 1 round and λf bits of communication in the offline phase and 2 rounds and $2^f + f$ bits of communication (across both parties) in the online phase.

4.1.3 Secure membership test. In this protocol, we have a public set $\mathcal{Y} = \{0, \dots, T - 1\}$, P_0 has a private set $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}_T$ of size T^* , P_1 has a private element $y \in \mathcal{Y}$. Both parties would like to securely compute (a Boolean sharing of) a bit b indicating whether $y \in \mathcal{Y}_{T^*}$.

Protocol. We describe a protocol for realizing this task in Figure 5.

The idea behind the protocol is simple: P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_2}$ where P_0 acts as a sender and sets the i^{th} OT sender string to be 1 iff $i \in \mathcal{Y}_{T^*}$. Then P_1 simply acts as the OT receiver using y as its choice index. It is easy to see that the output of P_1 will be 1 iff $y \in \mathcal{Y}_{T^*}$. In order to have the result of membership test be in a secret-shared form (instead of revealing it in clear to P_1), P_0 can simply mask its OT sender strings using a random bit z and output z as its share of the result. The OT receiver output of the P_1 together with the output of P_0 (which is z) will then constitute a Boolean sharing of the membership test result.

Security and Efficiency. To formally prove the security of $\Pi_{\text{membership}}$, we define an ideal functionality $\mathcal{F}_{\text{membership}}$ (Figure 10 in Appendix D.1) and then show that $\Pi_{\text{membership}}$ securely realizes $\mathcal{F}_{\text{membership}}$ in the $\mathcal{F}_{\text{OT}}^{(T), \mathbb{Z}_2}$ hybrid model (Theorem D.1 in Appendix D.1). In

Protocol: $\Pi_{\text{membership}}$

Public input: Set $\mathcal{Y} = \{0, \dots, T-1\}$, where $T \in \mathbb{N}$.
 Private inputs: P_0 holds $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}$ where $T^* = |\mathcal{Y}_{T^*}|$ and P_1 holds $y \in \mathcal{Y}$.
 Output: $\langle b \rangle$ where $b := \mathbf{1}\{y \in \mathcal{Y}_{T^*}\}$

- (1) P_0 samples $z \leftarrow \{0, 1\}$.
- (2) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_2}$:
 - P_0 acts as the OT sender and sets the i^{th} OT bit $s_i = \mathbf{1}\{i \in \mathcal{Y}_{T^*}\} \oplus z$
 - P_1 acts as the OT receiver with the choice index y .
 - P_1 retrieves s_y .
- (3) P_0 outputs z .
- (4) P_1 outputs s_y .

Figure 5: Protocol $\Pi_{\text{membership}}$ for secure membership test.

terms of efficiency, the protocol performs a single call to the OT functionality $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_2}$ which incurs 1 round and $\lambda \log T$ bits of communication in the offline phase and 2 rounds and $T + \log T$ bits of communication (across both parties) in the online phase.

4.1.4 4-choose-1 secure multiplexer. In this protocol, both parties hold an arithmetic sharing of four values, $(x^{0,0}, x^{0,1}, x^{1,0}, x^{1,1})$, over $\mathbb{Z}_T$ and a Boolean sharing of two choice bits (c, d) . They would like to securely compute an (fresh) arithmetic sharing of $x^{c,d}$.

Protocol. We describe a protocol $\Pi_{\text{mux}}^{(1)}$ for realizing this task using $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ in Figure 14 in Appendix D.4, which follows from [63].

Security and Efficiency. To formally prove the security of $\Pi_{\text{mux}}^{(1)}$, we define an ideal functionality $\mathcal{F}_{\text{mux}}^{(1)}$ (Figure 13, Appendix D.4) and then state that $\Pi_{\text{mux}}^{(1)}$ securely realizes $\mathcal{F}_{\text{mux}}^{(1)}$ in the $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ hybrid model (Theorem D.4 in Appendix D.4). In terms of efficiency, the protocol performs two calls to the OT functionality $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ which incurs 1 round and 4λ bits of communication in the offline phase and 2 rounds (since the two OTs can be executed in parallel) and $2(4 \log T + 2)$ bits of communication (across both parties) in the online phase.

4.2 Overall Protocol

We will now describe the overall protocol $\Pi_{\text{RRwithPrior}}$ for realizing $\mathcal{F}_{\text{RRwithPrior}}$. To do so, we will assume that parties have access to functionalities $\mathcal{F}_*$ corresponding to the subprotocols Π_* where $*$ $\in \{\text{rand}, \text{membership}, \text{biasBit}, \text{mux}\}$. These functionalities are described in Appendix D.

Protocol. Figure 6 describes our protocol $\Pi_{\text{RRwithPrior}}$ for securely realizing the $\mathcal{F}_{\text{RRwithPrior}}$ functionality introduced earlier. It is a simple combination of the modular sub-protocols as we described earlier in Table 1 followed by a final reconstruction of the perturbed label $\tilde{y}$ in the last step towards P_0 . The correctness of the overall protocol follows directly from the correctness of the underlying sub-protocols.

Security and Efficiency. To formally prove the security of $\Pi_{\text{RRwithPrior}}$, we show (Theorem D.6 in Appendix D.5) that $\Pi_{\text{RRwithPrior}}$ (Figure 6) securely realizes $\mathcal{F}_{\text{RRwithPrior}}$ (Figure 2) in the $\mathcal{F}_{\text{membership}}, \mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{mux}}^{(1)}, \mathcal{F}_{\text{rand}}$ hybrid model, with respect to passive corruption. The efficiency of $\Pi_{\text{RRwithPrior}}$ will be governed by the efficiency of the underlying sub-protocols namely $\Pi_{\text{membership}}, \Pi_{\text{biasBit}}, \Pi_{\text{mux}}^{(1)}, \Pi_{\text{rand}}$. As each of these sub-protocols is invoked exactly once in $\Pi_{\text{RRwithPrior}}$, the communication cost is simply the sum of the communication cost of each sub-protocol. Therefore, the offline and online communication cost of $\Pi_{\text{RRwithPrior}}$ comes out to be $\lambda(3 \log T + f + 4)$ and $\log T(2T + 11) + T + 2^f + f + 4$ bits respectively. In terms of rounds, the offline phase requires a single round as all the sub-protocols just require a single offline round for OT extension which can be performed in parallel.

Protocol: $\Pi_{\text{RRwithPrior}}$

Public inputs: Number of classes T , label set $\mathcal{Y} = \{0, \dots, T-1\}$, privacy parameter $\epsilon \in \mathbb{R}_{\geq 0}$, fixed point precision $f \in \mathbb{N}$, arithmetic sharing domain $\mathbb{Z}_T$.
 Private inputs: P_0 holds a prior vector $\vec{p} = \{p_i\}_{i \in [T]}$ where $p_i \in \mathbb{R}, p_i \in [0, 1]$ and $\sum_{i \in [T]} p_i = 1$. P_1 holds a label $y \in \mathcal{Y}$.
 Output: P_0 outputs a perturbed label $\tilde{y} \in \mathcal{Y}$.

- (1) P_0 computes $T^* = \arg\max_{t \in [T]} \left(\frac{e^\epsilon}{e^\epsilon + t - 1} \cdot (\sum_{i \in \mathcal{Y}_t} p_i) \right)$, where $\mathcal{Y}_t$ is the set of t labels with maximum p_i values.
- (2) P_0 sets $\mathcal{Y}_{T^*}$ to be the set of T^* labels with maximum p_i values.
- (3) P_0 computes $q_{\text{fix}} = \left\lfloor \frac{e^\epsilon - 1}{e^\epsilon + T^* - 1} \cdot 2^f \right\rfloor$.
- (4) P_0 and P_1 invoke $\mathcal{F}_{\text{biasBit}}$, where P_0 inputs q_{fix} , and they receive shares $\llbracket b_1 \rrbracket_0$ and $\llbracket b_1 \rrbracket_1$ respectively.
- (5) P_0 and P_1 invoke $\mathcal{F}_{\text{membership}}$, where P_0 inputs $\mathcal{Y}_{T^*}$ and P_1 inputs y , and they receive shares $\llbracket b_2 \rrbracket_0$ and $\llbracket b_2 \rrbracket_1$ respectively.
- (6) P_0 and P_1 invoke $\mathcal{F}_{\text{rand}}$, where P_0 inputs $\mathcal{Y}_{T^*}$, and they receive shares $\llbracket z \rrbracket_0$ and $\llbracket z \rrbracket_1$ respectively.
- (7) P_0 and P_1 set $\llbracket y \rrbracket_0 = 0 \in \mathbb{Z}_T$ and $\llbracket y \rrbracket_1 = y \in \mathbb{Z}_T$ respectively, where $\llbracket y \rrbracket = (\llbracket y \rrbracket_0, \llbracket y \rrbracket_1)$ denotes a default sharing of the value y over $\mathbb{Z}_T$ held by P_1 .
- (8) P_0 and P_1 invoke $\mathcal{F}_{\text{mux}}^{(1)}$ using selection shares $(\llbracket b_1 \rrbracket, \llbracket b_2 \rrbracket)$ and value shares $(\llbracket z \rrbracket, \llbracket z \rrbracket, \llbracket y \rrbracket)$. They finally receive a sharing of $\llbracket \tilde{y} \rrbracket$, where $\tilde{y} = y$ if $b_1 \wedge b_2 = 1$, and z otherwise.
- (9) P_1 sends $\llbracket \tilde{y} \rrbracket_1$ to P_0 . P_0 then reconstructs $\tilde{y} := \llbracket \tilde{y} \rrbracket_0 + \llbracket \tilde{y} \rrbracket_1 \pmod T$ and outputs it.

Figure 6: Protocol $\Pi_{\text{RRwithPrior}}$ for securely computing $\mathcal{F}_{\text{RRwithPrior}}$.

For online rounds, we note that the sub-protocols for $\mathcal{F}' = \{\mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{membership}}, \mathcal{F}_{\text{rand}}\}$ can be invoked in parallel as their inputs and outputs do not have any inter-dependency. However, the sub-protocol for $\mathcal{F}_{\text{mux}}^{(1)}$ can only be invoked after the sub-protocols for $\mathcal{F}'$ are completed because the output of $\mathcal{F}'$ is fed as input to $\mathcal{F}_{\text{mux}}^{(1)}$. This means that the total online rounds for $\Pi_{\text{RRwithPrior}}$ will be 5 as we need maximum 3 online rounds for the sub-protocols for $\mathcal{F}'$ and 2 online rounds for the sub-protocol for $\mathcal{F}_{\text{mux}}^{(1)}$. As an optimization, we note that this can be reduced to 4 rounds instead by parallelizing the third round of Π_{rand} with the first round of $\Pi_{\text{mux}}^{(1)}$.

Protocol/Costs	Online communication (bits)	Online rounds	Offline communication (bits)
$\Pi_{\text{membership}}$	$T + \log T$ 14	2	$\lambda \log T$ 512
Π_{rand}	$2(T + 1) \log T$ 88	3	$2\lambda \log T$ 1024
Π_{biasBit}	$2^f + f$ 1034	2	λf 1280
$\Pi_{\text{mux}}^{(4)}$	$8 \log T + 4$ 36	2	4λ 512
$\Pi_{\text{RRwithPrior}}$	$\log T(2T + 11) + T + 2^f + f + 4$ 1172	4	$\lambda(3 \log T + f + 4)$ 3328

Table 2: Analytical costs of $\Pi_{\text{RRwithPrior}}$ sub-protocols. The blue highlighted numbers indicate concrete costs for $T = 10$, $\lambda = 128$ and $f = 10$. All protocols require a single offline round (due to OT extension). The costs presented here do not include the cost of “base OTs” as it is amortized away over large number of OT instances. The logarithm function $\log$ is w.r.t base 2.

This is possible because the first round of $\Pi_{\text{mux}}^{(4)}$ is only dependent on the output of sub-protocols Π_{biasBit} , Π_{interval} , both of which are completed by the second round. As a summary, we present the analytical costs of all our protocols in Table 2.

THEOREM 4.1 (SECURE RRWithPrior). *Let T be the number of possible labels, f be a fixed point parameter and λ be a security parameter. There exists a protocol $\Pi_{\text{RRwithPrior}}$ (Fig.6) securely realizes $\mathcal{F}_{\text{RRwithPrior}}$ (Fig.2) in the $\mathcal{F}_{\text{OT}}$ hybrid model against semi-honest adversaries using $\approx 2T \log T + 2^f$ bits of online communication and 4 online rounds. Using OT-extension to instantiate $\mathcal{F}_{\text{OT}}$, we have $\approx \lambda(3 \log T + f)$ bits of offline communication and a single offline round.¹⁰*

Proof. We refer the readers to Section 4.2 for efficiency analysis. For security, we take the following approach. First, we define functionalities $\mathcal{F}_*$ in Appendix D corresponding to each of the sub-protocols Π_* where $*$ $\in \{\text{rand, membership, sample-bias-bit, mux}\}$ and show that Π_* securely realizes the functionality $\mathcal{F}_*$ in the $\mathcal{F}_{\text{OT}}$ hybrid model. We then show (Theorem D.6, Appendix D) that $\Pi_{\text{RRwithPrior}}$ (Figure 6) securely realizes $\mathcal{F}_{\text{RRwithPrior}}$ (Figure 2) in the $\mathcal{F}_{\text{membership}}, \mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{mux}}^{(4)}, \mathcal{F}_{\text{rand}}$ hybrid model. By invoking the UC composability theorem [18], it follows that $\Pi_{\text{RRwithPrior}}$ securely realizes $\mathcal{F}_{\text{RRwithPrior}}$ in the $\mathcal{F}_{\text{OT}}$ hybrid model if the calls to $\mathcal{F}_*$ are replaced by calls to Π_* for $*$ $\in \{\text{rand, membership, sample-bias-bit, mux}\}$.

Note that RRWithPrior mechanism is aimed towards adding Label DP guarantee for model training on classification tasks. Ghazi et al.[38] recently proposed a variant called Randomized Response on Bins (RRonBins) which handles regression tasks. While the focus of this work is specifically on RRWithPrior, our techniques can also be extended to securely compute RRonBins. Due to space limitation, we refer the readers to Appendix C for details.

5 Experimental Evaluation

Implementation Details. We implement our system in C++ and Python 3. Our system implements the proposed secure protocol $\Pi_{\text{RRwithPrior}}$ (Figure 6) and applies it to train neural networks with LabelDP using LP-MST (Algorithm 2). C++ coordinates the whole computation and executes the 2PC protocols. At each LP-MST iteration, the C++ engine makes two invocations to Python scripts that use the Keras [22] interface for neural networks with TensorFlow [15] backend: (1) forward propagation to compute the priors

(Line 6 in Algorithm 2), and (2) model training on the randomized labels (Line 9 in Algorithm 2). We embed Python in C++ using C++’s default application programmer’s interface to Python (see the Python.h header). This architecture enables us to use Python’s convenient libraries for deep learning and linear algebra, including Keras and TensorFlow, and support any (Keras-implemented) model. We note some interesting aspects of our implementation:

- We store the neural network architecture, the model weights, and the training configuration (e.g. loss function, optimizer, etc.) in HDF5 file format. Keras’ API provides functions to load from and save to this format. Note that this approach is highly modular as it allows us to easily switch to a completely different neural network without having to modify code.
- In our codebase, we include an efficient implementation of IKNP OT extension [47], including the state-of-the-art optimizations [5, 43]. We precompute a large number of random 1-out-of-2 OTs in the offline phase, and convert them into 1-out-of- N random OTs in a straightforward way via $\log N$ 1-out-of-2 random OTs. In the online phase, we then construct 1-out-of- N chosen OTs from the precomputed 1-out-of- N random OTs with Beaver’s classic trick [7].
- Our system includes server/client code with network communication implemented using gRPC’s asynchronous APIs.
- Our implementation has end-to-end tests written in the GoogleTest library [42].
- Finally, we use the Bazel build system [41].

Datasets. We run our experiments on the following datasets:

- **MNIST [26]** is a dataset of hand-written digits classified into one of 10 classes, namely 0-9. Each example, representing a hand-written digit, is a 28×28 gray scale image (i.e. 784 features). The training set contains 60,000 examples; the testing set contains 10,000.
- **CIFAR-10 [53]** is an image classification dataset. There are 10 classes corresponding to different objects such as automobile, airplane, frog, dog, etc. Each dataset example is a 32×32 color image with red, green, and blue channel values (i.e. 3072 features). The training set contains 50,000 examples; the testing set contains 10,000.

Network Architecture and Training Configuration. For MNIST, we use a simple convolutional neural network (CNN) that starts with two convolutional layers, each followed by max-pooling to reduce the spatial dimensions of the feature maps. After flattening the output, we use a dropout layer to reduce overfitting, and apply a dense softmax layer to predict one of the 10 digit classes. We compile the model using categorical cross-entropy loss and the Adam

¹⁰This excludes the cost of “base OTs” as it is amortized away over large number of OT instances.

	MNIST (15 Keras epochs)			CIFAR-10 (35 Keras epochs)			
Fixed-point precision f Accuracy	$\epsilon = 1$						
	12 93.8% (93.7%)			13 52.6 % (53.0%)			
	<u>Offline</u>	<u>Online</u>	<u>Total</u>	<u>Offline</u>	<u>Online</u>	<u>Total</u>	
	Communication	25.69 MB	237.00 MB	262.69 MB	22.18 MB	392.82 MB	415.00 MB
	LAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	19.12 sec —	15.32 sec —	34.44 sec 3.67 min	32.63 sec —	30.06 sec —	62.69 sec 33.27 min
	WAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	19.70 sec —	19.24 sec —	38.94 sec 3.75 min	33.42 sec —	42.36 sec —	75.78 sec 33.49 min
Fixed-point precision f Accuracy	$\epsilon = 3$						
	10 98.5% (98.6%)			12 88.1% (88.7%)			
	<u>Offline</u>	<u>Online</u>	<u>Total</u>	<u>Offline</u>	<u>Online</u>	<u>Total</u>	
	Communication	23.86 MB	61.22 MB	85.08 MB	21.42 MB	197.50 MB	218.92 MB
	LAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	4.15 sec —	4.76 sec —	8.91 sec 3.25 min	16.09 sec —	14.69 sec —	30.78 sec 32.74 min
	WAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	4.82 sec —	6.64 sec —	11.46 sec 3.29 min	16.82 sec —	19.24 sec —	36.06 sec 32.83 min
Fixed-point precision f Accuracy	$\epsilon = 8$						
	8 99.1% (99.1%)			10 92.8% (93.5%)			
	<u>Offline</u>	<u>Online</u>	<u>Total</u>	<u>Offline</u>	<u>Online</u>	<u>Total</u>	
	Communication	22.03 MB	17.26 MB	39.29 MB	19.89 MB	51.02 MB	70.91 MB
	LAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	1.18 sec —	2.24 sec —	3.42 sec 3.16 min	3.67 sec —	4.62 sec —	8.29 sec 32.37 min
	WAN $\left\{ \begin{array}{l} \text{2PC Only} \\ \text{Full Training} \end{array} \right.$	1.77 sec —	3.72 sec —	5.49 sec 3.19 min	4.49 sec —	7.02 sec —	11.51 sec 32.42 min

Table 3: We benchmark LP-MST for different ϵ (DP privacy budget) and f (fixed-point precision). We fix the number of LP-MST iterations to 2. We compare test dataset accuracy of LP-MST with $\Pi_{RRwithPrior}$ against LP-MST with plaintext RRWithPrior (in parentheses). We present runtime (LAN and WAN) and communication for the offline phase, online phase, and total (offline + online). In all executions, offline phase uses 2 rounds of interaction, while the online phase requires 8. All wall-clock times represent the *maximum* of the client and server; all communication costs represent the *sum* of the client and server. “2PC Only” represents the running time overhead of only $\Pi_{RRwithPrior}$; “Full Training” represents the total runtime of LP-MST including $\Pi_{RRwithPrior}$.

optimizer. We train the model over 15 epochs and a batch size of 128. We reserve 6,000 images (10% of the official 60,000-image training split) as a validation set, used solely to monitor generalization: validation loss and accuracy are logged after each epoch but never influence gradients, early stopping, or hyperparameter tuning. For CIFAR-10, we implement a ResNet-18 V2 architecture with data augmentation and a custom learning rate scheduler. We start with convolutional layers, followed by ResNet blocks that downsample the spatial dimensions while increasing filter counts, and we conclude with global average pooling and a softmax output for classification. To improve generalization, we apply data augmentation techniques such as random horizontal flips and width/height shifts. We train the model using the SGD optimizer with a momentum

of 0.9 and an initial learning rate of 0.4, which decays over time through a custom learning rate schedule. The learning rate ramps up during the first 30 epochs before decaying in later stages. We train the model for 35 epochs with a batch size of 512. For all experiments, we use 2 LP-MST iterations and split the dataset equally between these iterations.

Experimental setup. We run our MNIST experiments on two Ubuntu 20.04 LTS compute-optimized c2-standard-8 Google Cloud instances with 32 GB RAM and Intel Xeon CPU at 3.1 GHz clock rate. For CIFAR-10, the machine that runs the deep learning training and prior computation is replaced with an Ubuntu 20.04 LTS g2-standard-32 Google Cloud instance with a single NVIDIA L4 GPU, 128GB RAM, and Intel Xeon CPU at 2.2GHz clock rate. In the LAN

Fixed-point precision (f)		8	10	15	20
Communication (KB)	Offline	0.43 (218.2×)	0.46 (211.5×)	0.54 (193.3×)	0.62 (179.9×)
	Online	0.34 (5.2×)	1.09 (1.7×)	32.09 (16×)	1024.09 (468.2×)
	Total	0.77 (124.5×)	1.55 (64.4×)	32.63 (3.3×)	1024.71 (9×)
Rounds	Offline	2 (14×)	2 (14×)	2 (14×)	2 (14×)
	Online	4 (11.8×)	4 (11.8×)	4 (12.3×)	4 (12.3×)
	Total	6 (12.5×)	6 (12.5×)	6 (12.8×)	6 (12.8×)
LAN runtime (ms)	Offline	0.12 (8×)	0.18 (5.6×)	2.78 (2.4×)	102.28 (79.3×)
	Online	2.11 (2.9×)	2.21 (1.7×)	4.54 (1.4×)	88.80 (14.5×)
	Total	2.23 (3.2×)	2.39 (2×)	7.32 (1×)	191.08 (25.8×)
WAN runtime (ms)	Offline	0.50 (118.3×)	0.59 (108.2×)	3.17 (23.2×)	102.33 (1.3×)
	Online	315.08 (3.4×)	320.66 (3.3×)	365.97 (3×)	661.62 (1.7×)
	Total	315.58 (3.6×)	321.25 (3.5×)	369.14 (3.2×)	763.95 (1.6×)

Table 4: This table shows the concrete cost of a single execution of our $\Pi_{\text{RRwithPrior}}$ protocol for different values of the fixed-point precision (f). We also show the improvement and degradation factor, w.r.t. a baseline approach, indicated by green and red numbers respectively within parentheses. The baseline approach refers to a protocol execution using the semi-honest version of SPDZ_{2k} protocol implemented in the MP-SPDZ [51]. The communication cost is across both parties (client and server) combined. The offline cost is amortized over 1000 executions.

setting, we deploy both instances in the us-central1 region, with 0.12ms mean network latency and $\approx 1.9\text{GB/s}$ bandwidth. In the WAN setting, we deploy one instance in us-central1 and the other in us-west1. The mean network latency is 31.30ms and the bandwidth $\approx 93\text{MB/s}$. The client and the server execute their computation simultaneously in each round. They proceed to the next round once both parties have completed the computation and transferred the messages to one another. As LP-MST is a randomized algorithm (due to RRWithPrior and Keras model training), we take mean averages over 10 runs for more consistent results.

Goal of our evaluation. [37] already demonstrates that RRWithPrior provides more accurate results than vanilla RR (and other approaches) while maintaining the same level of privacy. We do not repeat their experiments. The goal of our evaluation is to show that RRWithPrior, when replaced with our proposed secure version $\Pi_{\text{RRwithPrior}}$, does not introduce significant runtime overhead or accuracy loss in the context of LP-MST training. In other words, given a fixed model architecture, using $\Pi_{\text{RRwithPrior}}$ to randomize the labels (rather than randomizing them in cleartext using RRWithPrior) does not significantly reduce the overall training accuracy or increase the running time of LP-MST. Hence, we do not try to find the most performant network architecture and training parameters (which is purely an ML task). We also note that the running time and communication cost of our $\Pi_{\text{RRwithPrior}}$ is independent of the model as $\Pi_{\text{RRwithPrior}}$ is only used to randomize each label in the dataset exactly once.

LP-MST accuracy experiments. We now show that $\Pi_{\text{RRwithPrior}}$ does not significantly degrade the accuracy of LP-MST w.r.t. plaintext RRWithPrior. Table 3 shows that for all experiments the accuracy differs by at most 1%. Note that a larger number of fractional bits f can decrease this difference at the expense of increasing the runtime and communication cost (as we will discuss next).

LP-MST runtime and communication experiments. As shown in the ‘2PC Only’ rows of Table 2, the cost of $\Pi_{\text{RRwithPrior}}$ during full training in our most computationally intensive CIFAR-10 experiment is less than 1.3 minutes, whereas the total training time

exceeds 30 minutes. Our 2PC requires 2 rounds of interaction to pre-process OTs in the offline phase and 4 rounds per online iteration, which is consistent with our analytical estimates in Table 2.

Scalability of our experiments. Note that the ML training happens outside 2PC on the server side on the noisy labels received from each client. The only part executed inside 2PC is the prior-based RR mechanism, which securely computes and releases the noisy labels to the server. Beyond this one-shot 2PC interaction between each client and the server, the remaining computation on the server side is identical to the traditional ML training on plaintext (but DP perturbed) labels. This leads to the following conclusions: i) Each client’s communication/computation cost is independent of the total number of clients in the system. However, the server’s communication/computation cost across all clients scales linearly with the number of clients as expected. ii) Since the number of clients is identical to the training set size (as each client is holding a single label¹¹), the cost dependency on the training set size is identical to the cost dependency on the number of clients. iii) The size/complexity of the ML model does not affect the 2PC communication/computation cost of the server. It only affects the running time and memory consumption of the “non-2PC” part, which is ML training, in the same way that the model size would affect the time/memory cost of vanilla ML training.

To illustrate the scalability of $\Pi_{\text{RRwithPrior}}$ we perform a batch of 10^6 executions of $\Pi_{\text{RRwithPrior}}$ protocol. This corresponds to 10^6 clients, each holding a single label, interacting with the server to randomize their labels securely. For $f = 10$, $T = 10$, this requires 366 MB (resp. 288 MB) of communication in the offline (resp. online) phases. In terms of runtime, we get 20 s (resp. 37 s) on LAN in the offline (resp. online) phases and 26 s (resp. 44 s) on WAN in the offline (resp. online) phases.

Microbenchmarking $\Pi_{\text{RRwithPrior}}$. In Table 4, we show the concrete cost of a single execution of the $\Pi_{\text{RRwithPrior}}$ protocol for different values of fixed-point precision (f). Although this work is the

¹¹ Our techniques are not restricted to the assumption that the client holds a single label. Specifically, if a client holds k labels, they would perform k independent executions of the protocol $\Pi_{\text{RRwithPrior}}$ with the server, where each execution would securely output a randomized/noisy version of the client’s label to the server.

first to address the question of efficiently computing RRWithPrior using 2PC, we can compare our protocol against a baseline approach using generic 2PC techniques known in the literature. To that end, we establish a tight baseline by executing the RRWithPrior computation, as described in Algorithm 1 (i.e. without our simplification in Sec 4), via semi-honest version of the popular and highly performant SPDZ protocol [23, 24] over $\mathbb{Z}_{2^k}$ ring as implemented (by the name “semi2k”) in the widely used MP-SPDZ library [51]. We use a standard 64 bit ring along with the local share conversion optimization as suggested by the MP-SPDZ compiler. We use this implementation to benchmark the concrete cost of the baseline approach on the same experimental setup as our approach and highlight the improvement/degradation factor of our approach (w.r.t. baseline) in Table 4 .

We note that as f increases, the *online* communication cost of our approach increases, thus reducing our improvement factor w.r.t the baseline approach. However, as indicated by the accuracy values in Table 3, values of $f \in [8, 15]$ suffice to get good accuracy (compared to the insecure plaintext approach). For these values of f , our approach offers $3.3 \times -124.5 \times$ improvement in total communication cost, $\approx 12.5 \times$ improvement in total rounds, $1 \times -3.2 \times$ improvement in total LAN runtime and $3.2 \times -3.6 \times$ improvement in total WAN runtime.

Impact of fixed-point arithmetic on DP. Recent works have shown that implementations of Differential Privacy mechanisms require care to prevent against leakage due to imprecision of concrete numerical implementations. Mironov [59] was the first to show concrete attacks for mechanisms such as the Laplace distribution, relying on concrete floating point implementations. Many other works [6, 36, 45, 46, 49] have explored this space presenting both attacks and mitigations.

We point out that our approaches are exempt from the attacks on floating point implementations described above since they make use of Bernoulli sampling, namely flipping a coin with probability p in Π_{biasBit} , or sampling at random from T classes in Π_{rand} . However, even Bernoulli sampling can be subject to bias in a fixed-point implementation. For Π_{rand} we avoid this bias by having each party sample a large random string (more than 40 bits longer than $\log(T)$), and taking it modulo T . The result is 2^{-40} -statistically-close to a random sample modulo T .

In Π_{biasBit} we incur a small bias by our use of fixed-point precision of 20 in the representation of the probability p . This bias increases the effective epsilon by a small amount (less than 10^{-3} for the ϵ ’s considered in the paper), because the probability in the numerator in the DP expression may have increased by up to 2^{-20} , and that in the denominator reduced by 2^{-20} . We consider this small increase negligible. We note that it becomes more significant for $\epsilon > 12$, in which case we would need more than 20 bits.

Acknowledgments

We thank Lawrence Roy for helpful discussions.

References

- [1] 2023. Privacy-preserving metrics for Firefox. <https://divviup.org/blog/divvi-up-in-firefox/>.
- [2] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 308–318.
- [3] Jayadev Acharya, Kallista Bonawitz, Peter Kairouz, Daniel Ramage, and Ziteng Sun. 2020. Context Aware Local Differential Privacy. In *Proceedings of the 37th International Conference on Machine Learning*.
- [4] Nitin Agrawal, Ali Shahin Shamsabadi, Matt J. Kusner, and Adrià Gascón. [n. d.]. QUOTIENT: Two-Party Secure Neural Network Training and Prediction. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS 2019, London, UK, November 11–15, 2019, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.).
- [5] Gilad Asharov, Yehuda Lindell, Thomas Schneider, and Michael Zohner. 2013. More efficient oblivious transfer and extensions for faster secure computation. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 535–548.
- [6] Victor Balcer and Salil Vadhan. 2019. Differential Privacy on Finite Computers. *Journal of Privacy and Confidentiality* 9, 2 (2019).
- [7] Donald Beaver. 1995. Precomputing Oblivious Transfer. In *CRYPTO’95 (LNCS, Vol. 963)*, Don Coppersmith (Ed.). Springer, Heidelberg, 97–109. doi:10.1007/3-540-44750-4_8
- [8] James Henry Bell, Kallista A. Bonawitz, Adrià Gascón, Tancrede Lepoint, and Mariana Raykova. 2020. Secure Single-Server Aggregation with (Poly)Logarithmic Overhead. In *CCS*. ACM, 1253–1269.
- [9] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 1988. Completeness Theorems for Non-Cryptographic Fault-Tolerant Distributed Computation (Extended Abstract). In *20th ACM STOC*. ACM Press, 1–10. doi:10.1145/62212.62213
- [10] Ari Biswas and Graham Cormode. 2023. Interactive proofs for differentially private counting. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 1919–1933.
- [11] Andrea Bittau, Úlfar Erlingsson, Petros Maniatis, Ilya Mironov, Ananth Raghunathan, David Lie, Mitch Rudominer, Ushasree Kode, Julien Tinnés, and Bernhard Seefeld. 2017. Prochlo: Strong Privacy for Analytics in the Crowd. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28–31, 2017*.
- [12] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingberman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *MLSys*. mlsys.org.
- [13] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical Secure Aggregation for Privacy-Preserving Machine Learning. In *CCS*. ACM, 1175–1191.
- [14] Elette Boyle, Niv Gilboa, and Yuval Ishai. 2019. Secure computation with preprocessing via function secret sharing. In *Theory of Cryptography: 17th International Conference, TCC 2019, Nuremberg, Germany, December 1–5, 2019, Proceedings, Part I* 17. Springer, 341–371.
- [15] Google Brain. 2015. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. <https://www.tensorflow.org/>.
- [16] Gilles Brassard, Claude Crépeau, and Jean-Marc Robert. 1986. All-or-nothing disclosure of secrets. In *Conference on the Theory and Application of Cryptographic Techniques*. Springer, 234–238.
- [17] Robert Istvan Busa-Fekete, Andres Munoz Medina, Umar Syed, and Sergei Vasilevskii. 2023. Label differential privacy and private training data release. In *International Conference on Machine Learning*. PMLR, 3233–3251.
- [18] Ran Canetti. 2001. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*. IEEE, 136–145.
- [19] Jeffrey Champion, Abhi Shelat, and Jonathan Ullman. 2019. Securely sampling biased coins with applications to differential privacy. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 603–614.
- [20] Albert Cheu, Adam D. Smith, Jonathan Ullman, David Zeber, and Maxim Zhilyaev. 2019. Distributed Differential Privacy via Shuffling. In *EUROCRYPT 2019, Part I (LNCS, Vol. 11476)*, Yuval Ishai and Vincent Rijmen (Eds.). Springer, Heidelberg, 375–403. doi:10.1007/978-3-030-17653-2_13
- [21] Seung Geol Choi, Dana Dachman-Soled, S Dov Gordon, Linsheng Liu, and Arkady Yerukhimovich. 2022. Secure sampling with sublinear communication. In *Theory of Cryptography Conference*. Springer, 348–377.
- [22] François Chollet et al. 2015. Keras. <https://keras.io>.
- [23] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. 2018. SPDZ2k: Efficient MPC mod 2^k for Dishonest Majority. In *Annual International Cryptology Conference*. Springer, 769–798.
- [24] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. 2012. Multiparty Computation from Somewhat Homomorphic Encryption. In *Advances in Cryptology – CRYPTO 2012*, Reihaneh Safavi-Naini and Ran Canetti (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 643–662.
- [25] Sankha Das, Sayak Ray Chowdhury, Nishanth Chandran, Divya Gupta, Satya Lokam, and Rahul Sharma. 2024. Communication Efficient Secure and Private

- Multi-Party Deep Learning. *Cryptology ePrint Archive* (2024).
- [26] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142.
 - [27] Giovanni Di Crescenzo, Tal Malkin, and Rafail Ostrovsky. 2000. Single database private information retrieval implies oblivious transfer. In *Advances in Cryptology—EUROCRYPT 2000: International Conference on the Theory and Application of Cryptographic Techniques* Bruges, Belgium, May 14–18, 2000 *Proceedings* 19. Springer, 122–138.
 - [28] Diemert Eustache, Betlei Artem, Christophe Renaudin, and Amini Massih-Reza. 2018. A Large Scale Benchmark for Uplift Modeling. In *Proceedings of the AdKDD and TargetAd Workshop, KDD, London, United Kingdom, August, 20, 2018*. ACM.
 - [29] Cynthia Dwork, Krishnaram Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. 2006. Our data, ourselves: Privacy via distributed noise generation. In *Advances in Cryptology—EUROCRYPT 2006: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28–June 1, 2006. Proceedings* 25. Springer, 486–503.
 - [30] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *TCC 2006 (LNCS, Vol. 3876)*, Shai Halevi and Tal Rabin (Eds.). Springer, Heidelberg, 265–284. doi:10.1007/11681878_14
 - [31] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography: Third Theory of Cryptography Conference, TCC 2006, New York, NY, USA, March 4–7, 2006. Proceedings* 3. Springer, 265–284.
 - [32] Reo Eriguchi, Atsunori Ichikawa, Noboru Kunihiro, and Koji Nuida. 2022. Efficient noise generation protocols for differentially private multiparty computation. *IEEE Transactions on Dependable and Secure Computing* 20, 6 (2022), 4486–4501.
 - [33] Úlfar Erlingsson, Vitaly Feldman, Ilya Mironov, Ananth Raghunathan, Kunal Talwar, and Abhradeep Thakurta. 2019. Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity. In *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '19)*.
 - [34] Hossein Esfandiari, Vahab Mirrokni, Umar Syed, and Sergei Vassilvitskii. 2022. Label differential privacy via clustering. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 7055–7075.
 - [35] Mani Malek Esmaeili, Ilya Mironov, Karthik Prasad, Igor Shilov, and Florian Tramèr. 2021. Antipodes of Label Differential Privacy: PATE and ALIBI. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6–14, 2021, virtual*, Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (Eds.).
 - [36] Ivan Gazeau, Dale Miller, and Catuscia Palamidessi. 2016. Preserving differential privacy under finite-precision semantics. *Theoretical Computer Science* 655 (2016), 92–108.
 - [37] Badih Ghazi, Noah Golowich, Ravi Kumar, Pasin Manurangsi, and Chiyuan Zhang. 2021. Deep learning with label differential privacy. *Advances in neural information processing systems* 34 (2021), 27131–27145.
 - [38] Badih Ghazi, Pritish Kamath, Ravi Kumar, Ethan Leeman, Pasin Manurangsi, Avinash Varadarajan, and Chiyuan Zhang. 2022. Regression with Label Differential Privacy. *arXiv preprint arXiv:2212.06074* (2022).
 - [39] Oded Goldreich. 2009. *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge University Press.
 - [40] Xueluan Gong, Qian Wang, Yanjiao Chen, Wang Yang, and Xinchang Jiang. 2021. Model extraction attacks and defenses on cloud-based machine learning models. *IEEE Communications Magazine* 58, 12 (2021), 83–89.
 - [41] Google. 2022. Bazel. <https://bazel.build/>.
 - [42] Google. 2023. GoogleTest – Google Testing and Mocking Framework. <https://github.com/google/googletest>.
 - [43] Chun Guo, Jonathan Katz, Xiao Wang, and Yu Yu. 2020. Efficient and secure multiparty computation from fixed-key block ciphers. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 825–841.
 - [44] Gauri Gupta, Krithika Ramesh, Anwesh Bhattacharya, Divya Gupta, Rahul Sharma, Nishanth Chandran, and Rijurekha Sen. 2023. End-to-end Privacy Preserving Training and Inference for Air Pollution Forecasting with Data from Rival Fleets. In *Privacy Enhancing technologies Symposium (PETS) 2023*.
 - [45] Naoise Holohan and Stefano Braghin. 2021. Secure random sampling in differential privacy. In *Computer Security—ESORICS 2021: 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4–8, 2021, Proceedings, Part II* 26. Springer, 523–542.
 - [46] Christina Ilvento. 2020. Implementing the exponential mechanism with base-2 differential privacy. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 717–742.
 - [47] Yuval Ishai, Joe Kilian, Kobbi Nissim, and Erez Petrank. 2003. Extending oblivious transfers efficiently. In *Annual International Cryptology Conference*. Springer, 145–161.
 - [48] Yuval Ishai, Eyal Kushilevitz, Sigurd Meldgaard, Claudio Orlandi, and Anat Paskin-Cherniavsky. 2013. On the power of correlated randomness in secure computation. In *Theory of Cryptography: 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3–6, 2013. Proceedings*. Springer, 600–620.
 - [49] Jiankai Jin, Eleanor McMurtry, Benjamin IP Rubinstein, and Olga Ohrimenko. 2022. Are we there yet? timing and floating-point attacks on differential privacy systems. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 473–488.
 - [50] Hannah Keller, Helen Möllering, Thomas Schneider, Oleksandr Tkachenko, and Liang Zhao. 2024. Secure Noise Sampling for DP in MPC with Finite Precision. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*. 1–12.
 - [51] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 1575–1590.
 - [52] Joe Kilian. 1988. Founding cryptography on oblivious transfer. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*. 20–31.
 - [53] Alex Krizhevsky. 2009. CIFAR-10 (Canadian Institute for Advanced Research). (2009). <http://www.cs.toronto.edu/~kriz/cifar.html>
 - [54] Eyal Kushilevitz, Yehuda Lindell, and Tal Rabin. 2006. Information-theoretically secure protocols and security under composition. In *Proceedings of the thirty-eighth annual ACM Symposium on Theory of Computing*. 109–118.
 - [55] Eyal Kushilevitz and Rafail Ostrovsky. 1997. Replication is not needed: Single database, computationally-private information retrieval. In *Proceedings 38th annual symposium on foundations of computer science*. IEEE, 364–373.
 - [56] Jiacheng Liang, Ren Pang, Changjiang Li, and Ting Wang. 2024. Model extraction attacks revisited. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 1231–1245.
 - [57] Yehuda Lindell. 2017. How to simulate it—a tutorial on the simulation proof technique. *Tutorials on the Foundations of Cryptography: Dedicated to Oded Goldreich* (2017), 277–346.
 - [58] Mani Malek Esmaeili, Ilya Mironov, Karthik Prasad, Igor Shilov, and Florian Tramèr. 2021. Antipodes of label differential privacy: Pate and alibi. *Advances in Neural Information Processing Systems* 34 (2021), 6934–6945.
 - [59] Ilya Mironov. 2012. On significance of the least significant bits for differential privacy. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 650–661.
 - [60] Moni Naor and Benny Pinkas. 1999. Oblivious transfer and polynomial evaluation. In *Proceedings of the thirty-first annual ACM symposium on Theory of computing*. 245–254.
 - [61] PATCG. 2023. Private Conversion Optimisation. <https://github.com/patcg/meetings/issues/17>.
 - [62] Deevashwer Rathee, Anwesh Bhattacharya, Divya Gupta, Rahul Sharma, and Dawn Song. 2023. Secure Floating-Point Training. In *Usenix Security Symposium* 2023.
 - [63] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. CryptFlow2: Practical 2-Party Secure Inference. In *ACM CCS 2020*, Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna (Eds.). ACM Press, 325–342. doi:10.1145/3372297.3417274
 - [64] César Sabater, Florian Hahn, Andreas Peter, and Jan Ramon. 2023. Private sampling with identifiable cheaters. *Proceedings on Privacy Enhancing Technologies* 2023, 2 (2023).
 - [65] XU Shirong, Chendi Wang, Will Wei Sun, and Guang Cheng. 2023. Binary classification under local label differential privacy using randomized response mechanisms. *Transactions on Machine Learning Research* (2023).
 - [66] Tatsuya Takemura, Naoto Yanai, and Toru Fujiwara. 2020. Model extraction attacks on recurrent neural networks. *Journal of Information Processing* 28 (2020), 1010–1024.
 - [67] Kunal Talwar, Shan Wang, Audra McMillan, Vojta Jina, Vitaly Feldman, Bailey Basile, Aine Cahill, Yi Sheng Chan, Mike Chatzidakis, Junye Chen, Oliver Chick, Mona Chitnis, Suman Ganta, Yusuf Goren, Filip Granqvist, Kristine Guo, Frederic Jacobs, Omid Javidbakht, Albert Liu, Richard Low, Dan Mascenik, Steve Myers, David Park, Wonhee Park, Gianni Parsa, Tommy Pauly, Christian Priebe, Rehan Rishi, Guy Rothblum, Michael Scaria, Linmao Song, Congzheng Song, Karl Tarbe, Sebastian Vogt, Luke Winstrom, and Shundong Zhou. 2023. Samplable Anonymous Aggregation for Private Federated Data Analysis. *arXiv:2307.15017 [cs.CR]*
 - [68] Erik Taubeneck, Ben Savage, and Martin Thomson. 2022. Interoperable Private Attribution (IPA). <https://docs.google.com/document/d/1KpdSKD8-Rn0bWPTu4UtK54ks0yv2j22pA5SrAD9av4s>.
 - [69] Sameer Wagh, Divya Gupta, and Nishanth Chandran. 2019. SecureNN: 3-Party Secure Computation for Neural Network Training. *Proc. Priv. Enhancing Technol.* 2019, 3 (2019), 26–49.
 - [70] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *J. Amer. Statist. Assoc.* 60, 309 (1965), 63–69.
 - [71] Genqiang Wu, Yeping He, Jingzheng Wu, and Xianyao Xia. 2016. Inherit differential privacy in distributed setting: Multiparty randomized function computation. In *2016 IEEE Trustcom/BigDataSE/ISPA*. IEEE, 921–928.
 - [72] Andrew Chi-Chih Yao. 1986. How to Generate and Exchange Secrets (Extended Abstract). In *27th FOCS*. IEEE Computer Society Press, 162–167. doi:10.1109/SFCS.1986.25
 - [73] Sen Yuan, Milan Shen, Ilya Mironov, and Anderson Nascimento. 2021. Label private deep learning training based on secure multiparty computation and

A Semi-Honest Secure Two-Party Computation

The following description of semi-honest two-party computation is standard in the literature and is taken from [57].

Semi-honest adversary. The model that we consider in this work is that of two-party computation in the presence of *static semi-honest* adversaries. Such an adversary controls one of the parties (statically, and so at the onset of the computation) and follows the protocol specification exactly. However, it may try to learn more information than allowed by looking at the transcript of messages that it received and its internal state. A protocol that is secure in the presence of semi-honest adversaries guarantees that there is no inadvertent leakage of information. Semi-honest secure protocols are often designed as the first step towards achieving the stronger notions of malicious security.

Two-party computation (2PC). A two-party protocol problem is cast by specifying a possibly random process that maps pairs of inputs to pairs of outputs (one for each party). We refer to such a process as a *functionality* and denote it $f : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^* \times \{0, 1\}^*$, where $f = (f_1, f_2)$. That is, for every pair of inputs $x, y \in \{0, 1\}^*$, the output-pair is a random variable $(f_1(x, y), f_2(x, y))$ ranging over pairs of strings. The first party (with input x) wishes to obtain $f_1(x, y)$ and the second party (with input y) wishes to obtain $f_2(x, y)$.

Privacy by simulation. As expected, we wish to formalize the idea that a protocol is secure if whatever can be computed by a party participating in the protocol can be computed based on its input and output only. This is formalized according to the simulation paradigm by requiring the existence of a simulator who generates the view of a party in the execution. However, since the parties here have input and output, the simulator must be given a party's input and output in order to generate the view. Thus, security here is formalized by saying that a party's view in a protocol execution be simulatable given its input and output. This formulation implies that the parties learn nothing from the protocol execution beyond what they can derive from their input and prescribed output.

One important point to note is that since the parties are semi-honest, it is guaranteed that they use the actual inputs written on their input tapes. This is important since it means that the output is well-defined, and not dependent on the adversary. Specifically, for inputs x, y , the output is defined to be $f(x, y)$, and so the simulator can be given this value.

Definition of security. We begin with the following notation:

- Let $f = (f_1, f_2)$ be a probabilistic polynomial-time functionality and let π be a two-party protocol for computing f . (Throughout, whenever we consider a functionality, we always assume that it is polynomially-time computable.)
- The view of the i^{th} party ($i \in \{1, 2\}$) during an execution of π on (x, y) and security parameter λ is denoted by $\text{view}_i^\pi(x, y, \lambda)$ and equals $(w, r_i; m_1^i, \dots, m_t^i)$, where $w \in \{x, y\}$ (its input depending on the value of i), r_i equals the contents of the i^{th} party's internal random tape, and m_j^i represents the j^{th} message that it received.

- The output of the i^{th} party during an execution of π on (x, y) and security parameter λ is denoted by $\text{output}_i^\pi(x, y, \lambda)$ and can be computed from its own view of the execution. We denote the joint output of both parties by $\text{output}^\pi(x, y, \lambda) = (\text{output}_1^\pi(x, y, \lambda), \text{output}_2^\pi(x, y, \lambda))$.

Definition A.1 (Standalone security). Let $f = (f_1, f_2)$ be a functionality. We say that a protocol π securely computes f in the presence of static semi-honest adversaries in the standalone model if there exist probabilistic polynomial-time algorithms $\mathcal{S}_1$ and $\mathcal{S}_2$ such that

$$\begin{aligned} & \{(\mathcal{S}_1(1^\lambda, x, f_1(x, y)), f(x, y))\}_{x, y, \lambda} \\ & \stackrel{c}{=} \\ & \{(\text{view}_1^\pi(x, y, \lambda), \text{output}^\pi(x, y, \lambda))\}_{x, y, \lambda} \\ & , \text{ and} \\ & \{(\mathcal{S}_2(1^\lambda, x, f_2(x, y)), f(x, y))\}_{x, y, \lambda} \\ & \stackrel{c}{=} \\ & \{(\text{view}_2^\pi(x, y, \lambda), \text{output}^\pi(x, y, \lambda))\}_{x, y, \lambda} \end{aligned}$$

where $x, y \in \{0, 1\}^*$ such that $|x| = |y|$, $\lambda \in \mathbb{N}$, and $\stackrel{c}{=}$ denotes computational indistinguishability of the ensembles for all large enough values of λ .

REMARK A.1. We note that our protocols are: i) perfectly secure in the standalone model (i.e. in a model where the protocol is executed only once), ii) have straight-line black-box simulators, i.e., only assume oracle access to the corrupt party and do not rewind. Kushilevitz et al. [54] then implies security under general concurrent composition, which is equivalent to Universal Composable (UC) security [18].

REMARK A.2. We will sometimes prove that a protocol π UC securely realizes a functionality $\mathcal{F}$ in the $\mathcal{G}$ hybrid model. This means that the functionality $\mathcal{G}$ is available to the parties in the real-world. If ϕ is a protocol which UC securely realizes $\mathcal{G}$, then it follows by the UC composability theorem [18] that π UC securely realizes $\mathcal{F}$ in a world where calls to $\mathcal{G}$ are replaced by calls to ϕ .

B Differential Privacy and Label Differential Privacy

B.1 Differential Privacy

Differential Privacy (DP) provides security guarantees, which are orthogonal to the security guarantees provided by secure computation. At a high level, secure computation guarantees that the computation *process* (i.e. transforming the encrypted/secret-shared input x to the output $f(x)$) leaks no additional information besides $f(x)$. On the other hand, DP guarantees that the output $f(x)$ itself leaks no “sensitive” information about x . This is formalized in Definition B.2.

Definition B.1 (Dataset and Adjacency relationship for standard DP). Let $\mathcal{X}$ denote the universe of items. A dataset D is simply an ordered subset of $\mathcal{X}$. We sometimes parse a size n dataset D as $\{x_1, \dots, x_n\}$. We say that two datasets D and D' are *adjacent* if their size is same i.e. $|D| = |D'| = n$ and they are identical except for a

difference in a single item at some arbitrary index i.e. there exists a unique $i^* \in [n]$ s.t. $x_{i^*} \in D$, $x'_{i^*} \in D'$ and $x_{i^*} \neq x'_{i^*}$.

Definition B.2 (Differential Privacy [31]). Let $\epsilon, \delta \in \mathbb{R}_{\geq 0}$. A randomized algorithm $\mathcal{A}$ taking as input a dataset is said to be (ϵ, δ) -differentially private $((\epsilon, \delta)$ -DP) if for any two adjacent datasets D and D' (Definition B.1), and for any subset S of outputs of $\mathcal{A}$, it is the case that $\Pr[\mathcal{A}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{A}(D') \in S] + \delta$. If $\delta = 0$, then $\mathcal{A}$ is said to be ϵ -differentially private (ϵ -DP).

B.2 Label Differential Privacy

Label Differential Privacy (Label DP) is a relaxation of the standard DP, tailored towards some machine learning tasks, where the privacy guarantee is only required w.r.t dataset labels whereas the dataset features are considered non-sensitive. To formalize this, we need to define the notion of a dataset and an adjacency relationship between datasets.

Definition B.3 (Dataset and Adjacency relationship for Label DP). Let $\mathcal{X}$ and $\mathcal{Y}$ denote the feature and label set respectively. A dataset D is simply an ordered subset of $\mathcal{X} \times \mathcal{Y}$. We sometimes parse a size n dataset D as $\{(x_1, y_1), \dots, (x_n, y_n)\}$. We say that two datasets D and D' are *adjacent* if their size is same i.e. $|D| = |D'| = n$ and they are identical except for a difference in a single label at some arbitrary index i.e. there exists a unique $i^* \in [n]$ s.t. $(x_{i^*}, y_{i^*}) \in D$, $(x_{i^*}, y'_{i^*}) \in D'$ and $y_{i^*} \neq y'_{i^*}$.

Now that we defined the notion of a dataset and adjacency relationship, the Label DP notion is captured in Definition B.4.

Definition B.4 (Label Differential Privacy [37]). Let $\epsilon, \delta \in \mathbb{R}_{\geq 0}$. A randomized algorithm $\mathcal{A}$ taking as input a dataset is said to be (ϵ, δ) -label differentially private $((\epsilon, \delta)$ -label DP) if for any two adjacent training datasets D and D' (Definition B.3), and for any subset S of outputs of $\mathcal{A}$, it is the case that $\Pr[\mathcal{A}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{A}(D') \in S] + \delta$. If $\delta = 0$, then $\mathcal{A}$ is said to be simply ϵ -label differentially private (ϵ -LabelDP).

C Securely Computing RR on Bins

While the RRWithPrior protocol is aimed towards adding Label DP guarantee for model training on classification tasks, Ghazi et. al.[38] recently proposed a variant called Randomized Response on Bins (RRonBins) which handles regression tasks. We describe RRonBins in Algorithm 4 which makes black-box use of a function called ComputeOptimalPhi. Since the inner workings of the function ComputeOptimalPhi will not be relevant to us, we refer the readers to [38] for its full description. This function returns a finite set $\hat{\mathcal{Y}}$ and a non-decreasing mapping $\phi : \mathcal{Y} \rightarrow \hat{\mathcal{Y}}$ for a given label set $\mathcal{Y}$, prior p , privacy parameter ϵ and loss function ℓ . At a high-level, the inverse mapping ϕ^{-1} partitions the label set $\mathcal{Y}$ into a set of finite bins (exactly $|\hat{\mathcal{Y}}|$ many bins) and then the RRonBins algorithm simply performs vanilla RR on these bins, hence the name RRonBins. Ghazi et. al.[38] show that RRonBins satisfies ϵ -DP which is captured by the following theorem.

THEOREM C.1 ([38]). For all $y_{\min} \in \mathbb{R}$, $y_{\max} \in \mathbb{R}$, distribution p over $\mathcal{Y} = [y_{\min}, y_{\max}]$, $\epsilon \in \mathbb{R}_{\geq 0}$, loss function $\ell : \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$, RRonBins (Algorithm 4) is ϵ -DP (Definition B.2).

Algorithm 4: RRonBins

Parameters: Minimum and maximum label value $y_{\min} \in \mathbb{Z}$ and $y_{\max} \in \mathbb{Z}$ respectively, label set $\mathcal{Y} = [y_{\min}, y_{\max}] \subset \mathbb{R}$, privacy parameter $\epsilon \geq 0$, loss function $\ell : \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$, prior probability distribution p over $\mathcal{Y}$.

Inputs: Label $y \in \mathcal{Y}$.

Output: A perturbed label $\tilde{y} \in \mathcal{Y}$.

- 1 $(\hat{\mathcal{Y}}, \phi) := \text{ComputeOptimalPhi}(\mathcal{Y}, p, \epsilon, \ell)$, where $\hat{\mathcal{Y}} \subseteq [y_{\min}, y_{\max}]$ s.t. $|\hat{\mathcal{Y}}| = k$ is finite and $\phi : \mathcal{Y} \rightarrow \hat{\mathcal{Y}}$ s.t. ϕ is non-decreasing.
- 2 Let $\mathcal{P}_\phi = [b_0, b_1), \dots, [b_{k-1}, b_k)$ denote a partition of $\mathcal{Y}$ induced by ϕ^{-1} and $\hat{\mathcal{Y}} = \{\hat{y}_0, \dots, \hat{y}_{k-1}\}$ s.t. $b_0 = y_{\min}$, $b_k = y_{\max}$, and ϕ is constant over each partition, i.e., for all $i \in [0, k-1]$, $y' \in [b_i, b_{i+1})$, we have $\phi(y') = \hat{y}_i$.
- 3 Let $z' \in [0, k-1]$ indicate the partition index s.t. $b_{z'} \leq y < b_{z'+1}$.
- 4 Define a random variable Z with values in $[0, k-1]$ and let $\mathcal{D}$ be the induced probability distribution s.t.

$$\Pr[Z = z] = \begin{cases} \frac{e^\epsilon}{e^\epsilon + k - 1} & z = z' \\ \frac{1}{e^\epsilon + k - 1} & \text{otherwise} \end{cases}$$

- 5 Sample $\tilde{z} \leftarrow \mathcal{D}$ and set $\tilde{y} = \hat{y}_{\tilde{z}}$.
 - 6 **return** $\tilde{y}$.
-

We start by noting that our approach for securely computing RRonBins (Algorithm 4) will mirror the approach that we took in Section 4 for securely computing RRWithPrior. Since the problem formulation is same as before, we directly proceed to the next step where we simplify the sampling step in RRonBins (Algorithm 4) so that it is amenable to secure computation.

Simplifying the computation. In Algorithm 5, we describe an equivalent and simplified way of computing Steps 4-6 of Algorithm 4. At a high-level, this simplification basically represents the non-uniform distribution induced on $\tilde{y}$ in Algorithm 4 as a composition of uniform distribution and a biased bit (Bernoulli) distribution. It is easy to see that the induced probability distribution on $\tilde{y}$ in Algorithm 5 is identical to that of Algorithm 4. To show this, we will calculate the probability distribution of the random variable $\tilde{z}$ in Algorithm 5. First note that $\Pr[\tilde{z} = z'] = \Pr[b = 1] + \Pr[b = 0] \cdot \frac{1}{k} = q + \frac{1-q}{k} = \frac{e^\epsilon}{e^\epsilon + k - 1}$. Secondly, for any $u \neq z'$, we have $\Pr[\tilde{z} = u] = \Pr[b = 0] \cdot \frac{1}{k} = \frac{1}{e^\epsilon + k - 1}$. Hence, the probability distribution of $\tilde{z}$ (resp. $\tilde{y}$) in Algorithm 5 matches with that of $\tilde{z}$ (resp. $\tilde{y}$) in Algorithm 4.

Algorithm 5: Simplified sampling in RRonBins

- 1 Sample $b \leftarrow \text{Bernoulli}(q)$, where $q = \frac{e^\epsilon - 1}{e^\epsilon + k - 1}$.
 - 2 Sample $z \leftarrow [0, k-1]$
 - 3 If $b = 1$, set $\tilde{z} := z'$. Else set $\tilde{z} := z$.
 - 4 Set $\tilde{y} := \hat{y}_{\tilde{z}}$
 - 5 **return** $\tilde{y}$.
-

Defining the ideal functionality.

Now we define an ideal functionality $\mathcal{F}_{\text{RRonBins}}$ in Figure 7 which captures RRonBins (Algorithm 4) where we replace Steps 4-6 of Algorithm 4 with the simplified steps in Algorithm 5. The security of a protocol Π_{RRonBins} realizing $\mathcal{F}_{\text{RRonBins}}$ is captured via the

standard simulation based security notion which is described in Appendix A.

Protocol construction. We first note that Steps 3-6 in $\mathcal{F}_{\text{RRonBins}}$ can be computed locally by P_0 to derive the sets $\mathcal{P}_\phi, \hat{\mathcal{Y}}$ since it only depends on the prior p and public parameters. Also, Step 7 can be locally computed by P_1 since it only depends on its private label y . The main challenge lies in computing Steps 8 - 13 which requires the P_0 's sets $\mathcal{P}_\phi, \hat{\mathcal{Y}}$, along with its size k , to be kept private (in addition to the keeping P_1 's label y private). Once again, we will take a modular approach and design sub-protocols for individually computing each step, from Steps 8 - 13, securely. Fortunately, we will be able to reuse most of the sub-protocols we described in Section 4.1 including Π_{biasBit} , Π_{rand} and $\Pi_{\text{mux}}^{(4)}$ (we will actually replace $\Pi_{\text{mux}}^{(4)}$ with a simpler $\Pi_{\text{mux}}^{(2)}$ as we will need to select between two values instead of four). The only new protocol that we will need is Π_{interval} . Table 5 shows the mapping between the functionality steps and their corresponding sub-protocol names. Since Step 12-13 have a trivial protocol where both parties do a public reconstruction of $\tilde{z}$ (from Step 11) towards P_0 and then P_0 locally looks up $\hat{y}_{\tilde{z}}$ in its set $\hat{\mathcal{Y}}$, we do not explicitly introduce a protocol for these steps.

Functionality: $\mathcal{F}_{\text{RRonBins}}$

Public Parameters: Minimum and maximum label value $y_{\min} \in \mathbb{Z}$ and $y_{\max} \in \mathbb{Z}$ respectively, label set $\mathcal{Y} = [y_{\min}, y_{\max}) \subset \mathbb{R}$, privacy parameter $\epsilon \geq 0$, loss function $\ell : \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$, fixed-point precision $f \in \mathbb{N}$.

- (1) Get a prior distribution p over $\mathcal{Y}$ as input from P_0 .
- (2) Get a label $y \in \mathcal{Y}$ as input from P_1 .
- (3) Compute $(\hat{\mathcal{Y}}, \phi) := \text{ComputeOptimalPhi}(\mathcal{Y}, p, \epsilon, \ell)$, where $\hat{\mathcal{Y}} \subseteq [y_{\min}, y_{\max})$ s.t. $|\hat{\mathcal{Y}}| = k$ is finite and $\phi : \mathcal{Y} \rightarrow \hat{\mathcal{Y}}$ s.t. ϕ is non-decreasing.
- (4) Let $\mathcal{P}_\phi = [b_0, b_1), \dots, [b_{k-1}, b_k)$ denote a partition of $\mathcal{Y}$ induced by ϕ^{-1} and $\hat{\mathcal{Y}} = \{\hat{y}_0, \dots, \hat{y}_{k-1}\}$ s.t. $b_0 = y_{\min}$, $b_k = y_{\max}$, and ϕ is constant over each partition, i.e., for all $i \in [k]$, $y' \in [b_{i-1}, b_i)$, we have $\phi(y') = \hat{y}_i$.
- (5) Let $\mathcal{P}_{\phi, \text{fix}} = [b_{0, \text{fix}}, b_{1, \text{fix}}), \dots, [b_{k-1, \text{fix}}, b_{k, \text{fix}})$ where $b_{i, \text{fix}} = \lfloor b_i \cdot 2^f \rfloor$ for all $i \in [0, k]$.
- (6) Set $q_{\text{fix}} = \left\lfloor \frac{e^\epsilon - 1}{e^\epsilon + k - 1} \cdot 2^f \right\rfloor$.
- (7) Set $y_{\text{fix}} = \lfloor y \cdot 2^f \rfloor$.
- (8) Let $z' \in [0, k - 1]$ indicate the partition index s.t. $b_{z', \text{fix}} \leq y_{\text{fix}} < b_{z'+1, \text{fix}}$.
- (9) Sample $b \leftarrow \text{Bernoulli}(q_{\text{fix}}/2^f)$.
- (10) Sample $z \leftarrow [0, k - 1]$.
- (11) If $b = 1$, set $\tilde{z} := z'$. Else set $\tilde{z} := z$.
- (12) Set $\tilde{y} := \hat{y}_{\tilde{z}}$.
- (13) Send $\tilde{y}$ to P_0 .

Figure 7: Ideal functionality for RRonBins

Secure interval finding. In this protocol, we have a public set $S = [0, R) \subset \mathbb{N}$ where $R \in \mathbb{N}$. Furthermore, P_0 privately holds a list of k buckets/partitions $\mathcal{P} = \{[c_0, c_1), \dots, [c_{k-1}, c_k)\}$ where $c_0 = 0$ and $c_k = R$ and P_1 privately holds a value $d \in S$. Both parties would

Steps in $\mathcal{F}_{\text{RRonBins}}$	Sub-protocols
8	Π_{interval}
9	Π_{biasBit}
10	Π_{rand}
11	$\Pi_{\text{mux}}^{(2)}$

Table 5: Mapping between steps in $\mathcal{F}_{\text{RRonBins}}$ and the corresponding protocols.

like to securely compute (an additive sharing of) a partition index $z' \in [0, k - 1]$ s.t. $c_{z'} \leq d < c_{z'+1}$, while keeping the partition $\mathcal{P}$, along with its size k , private w.r.t. P_0 and value d private w.r.t. P_1 . We describe the protocol in Figure 8. The idea behind this protocol is simple: Both parties invoke $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ where P_0 acts as the OT sender and sets up the sender strings in such a way that the i^{th} sender string encodes (a masked version of) the partition number that i will fall into. When P_1 , acting as the OT receiver, uses its private value d as the OT choice index, it will retrieve (a sharing of) the partition number that d falls into.

Protocol: Π_{interval}

Public input: Public set $S = [0, R) \subset \mathbb{N}$ where $R \in \mathbb{N}$

Private inputs: P_0 holds a list of k buckets/partitions $\mathcal{P} = \{[c_0, c_1), \dots, [c_{k-1}, c_k)\}$ where $c_0 = 0$ and $c_k = R$. P_1 holds a value $d \in S$.

Output: $\llbracket z' \rrbracket$ where $z' \in [0, k - 1]$ s.t. $c_{z'} \leq d < c_{z'+1}$

- (1) P_0 samples $r \leftarrow \mathbb{Z}_R$.
- (2) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ OT:
 - P_0 acts as the OT sender and sets the i^{th} string (0 - indexed) as $s_i := k_i - r \pmod{R}$ where k_i is selected s.t. $c_{k_i} \leq i < c_{k_i+1}$.
 - P_1 acts the OT receiver using d as the choice index.
 - P_1 retrieves s_d and outputs it.
- (3) P_0 outputs r .

Figure 8: Protocol for secure interval finding.

To formally prove the security of Π_{interval} , we define an ideal functionality $\mathcal{F}_{\text{interval}}$ (Figure 17 in Appendix D.6) and then state that Π_{interval} securely realizes $\mathcal{F}_{\text{interval}}$ in the $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ hybrid model (Theorem D.7 in Appendix D.6). In terms of efficiency, we note that the protocol simply performs a single call to the OT functionality $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ which incurs 1 round and $\lambda \log R$ bits of communication in the offline phase and 2 rounds and $(R + 1) \log R$ bits of communication (across both parties) in the online phase.

Overall protocol. Figure 9 describes the overall protocol Π_{RRonBins} for realizing $\mathcal{F}_{\text{RRonBins}}$. To do so, we assume that parties have access to functionalities $\mathcal{F}_*$ corresponding to the subprotocols Π_* where $*$ $\in \{\text{rand}, \text{interval}, \text{biasBit}, \text{mux}\}$. These functionalities are described in Appendix D. The protocol Π_{RRonBins} is simple combination of the modular sub-protocols as we described earlier in Table 5 followed by a final reconstruction of the perturbed label

$\tilde{y}$ in the last step towards P_0 . To formally prove the security of Π_{RRonBins} , we state that Π_{RRonBins} securely realizes $\mathcal{F}_{\text{RRonBins}}$ in the $\mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{interval}}, \mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{mux}}^{(2)}$ -hybrid model.

THEOREM C.2. Π_{RRonBins} securely realizes $\mathcal{F}_{\text{RRonBins}}$ in the $\mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{interval}}, \mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{mux}}^{(2)}$ -hybrid model, with respect to a semi-honest adversary.

Proof. The proof follows directly by having the simulator control the hybrid functionalities in the ideal world.

Protocol: Π_{RRonBins}

Public Parameters: Minimum and maximum label value $y_{\min} \in \mathbb{Z}$ and $y_{\max} \in \mathbb{Z}$ respectively, label set $\mathcal{Y} = [y_{\min}, y_{\max}] \subset \mathbb{R}$, privacy parameter $\epsilon \geq 0$, loss function $\ell : \mathbb{R}^2 \rightarrow \mathbb{R}_{\geq 0}$, fixed-point precision $f \in \mathbb{N}$, fixed point range $R = \lfloor (y_{\max} - y_{\min}) \cdot 2^f \rfloor$.
Private inputs: P_0 holds a prior distribution p over $\mathcal{Y}$. P_1 holds a label $y \in \mathcal{Y}$.

Output: P_0 outputs a perturbed label $\tilde{y} \in \mathcal{Y}$.

- (1) P_0 computes $(\hat{\mathcal{Y}}, \phi) := \text{ComputeOptimalPhi}(\mathcal{Y}, p, \epsilon, \ell)$, where $\hat{\mathcal{Y}} \subseteq [y_{\min}, y_{\max}]$ s.t. $|\hat{\mathcal{Y}}| = k$ is finite and $\phi : \mathcal{Y} \rightarrow \hat{\mathcal{Y}}$ s.t. ϕ is non-decreasing.
- (2) P_0 sets $\mathcal{P}_\phi = \{[b_0, b_1), \dots, [b_{k-1}, b_k)\}$ to denote a partition of $\mathcal{Y}$ induced by ϕ^{-1} and $\hat{\mathcal{Y}} = \{\hat{y}_0, \dots, \hat{y}_{k-1}\}$ s.t. $b_0 = y_{\min}$, $b_k = y_{\max}$, and ϕ is constant over each partition, i.e., for all $i \in [k]$, $y' \in [b_{i-1}, b_i)$, we have $\phi(y') = \hat{y}_i$.
- (3) P_0 sets $\mathcal{P}_{\phi, \text{fix}} = \{[b_{0, \text{fix}}, b_{1, \text{fix}}), \dots, [b_{k-1, \text{fix}}, b_{k, \text{fix}})\}$ where $b_{i, \text{fix}} = \lfloor b_i \cdot 2^f \rfloor$ for all $i \in [0, k]$. P_0 sets $\mathcal{Q} = \{[c_0, c_1), \dots, [c_{k-1}, c_k)\}$ where $c_i = b_{i, \text{fix}} - b_{0, \text{fix}}$ for all $i \in [0, k]$.
- (4) P_0 sets $q_{\text{fix}} = \lfloor \frac{e^\epsilon - 1}{e^\epsilon + k - 1} \cdot 2^f \rfloor$.
- (5) P_1 sets $y_{\text{fix}} = \lfloor y \cdot 2^f \rfloor$, $y_{\min, \text{fix}} = \lfloor y_{\min} \cdot 2^f \rfloor$ and $d = y_{\text{fix}} - y_{\min, \text{fix}}$.
- (6) P_0 and P_1 invoke $\mathcal{F}_{\text{biasBit}}$, where P_0 inputs q_{fix} , and they receive shares $\llbracket b \rrbracket_0$ and $\llbracket b \rrbracket_1$ respectively.
- (7) P_0 and P_1 invoke $\mathcal{F}_{\text{interval}}$, with public parameter R , where P_0 inputs $\mathcal{Q}$ and P_1 inputs d , and they receive shares $\llbracket z \rrbracket_0$ and $\llbracket z' \rrbracket_1$ respectively.
- (8) P_0 and P_1 invoke $\mathcal{F}_{\text{rand}}$, with public set $\{0, \dots, R-1\}$, where P_0 inputs $\{0, \dots, k-1\} \subseteq \{0, \dots, R-1\}$, and they receive shares $\llbracket z \rrbracket_0$ and $\llbracket z \rrbracket_1$ respectively.
- (9) P_0 and P_1 invoke $\mathcal{F}_{\text{mux}}^{(2)}$ using selection shares $\llbracket b \rrbracket$ and value shares $(\llbracket z \rrbracket, \llbracket z' \rrbracket)$. They finally receive a sharing of $\llbracket \tilde{z} \rrbracket$ over $\mathbb{Z}_R$, where $\tilde{z} = z$ if $b = 0$, and z' otherwise.
- (10) P_1 sends $\llbracket \tilde{z} \rrbracket_1$ to P_0 .
- (11) P_0 reconstructs $\tilde{z} = \llbracket \tilde{z} \rrbracket_0 + \llbracket \tilde{z} \rrbracket_1 \pmod{R}$ and outputs $\tilde{y} := \hat{y}_{\tilde{z}}$.

Figure 9: Protocol for secure RRonBins.

D Functionalities and Security Proofs

We now describe the functionalities underlying the protocols that we present in this work. Then we state and prove theorems stating that our protocols securely realize these functionalities in the UC

model. For ease of exposition, we suppress the usage of session-ids in our functionality descriptions. Also, we will use $\mathcal{S}$ to denote the simulator (aka ideal-world adversary) in the functionalities.

D.1 Secure Membership Test

In Figure 10, we describe the ideal functionality for the secure membership test described in Section 4.1.

Functionality: $\mathcal{F}_{\text{membership}}$

Public Parameters: Set $\mathcal{Y} = \{0, \dots, T-1\}$, where $T \in \mathbb{N}$.

- (1) Get $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}$ as input from P_0 where $T^* = |\mathcal{Y}_{T^*}|$.
- (2) Get $y \in \mathcal{Y}$ as input from P_1 .
- (3) Compute a bit $b := \mathbf{1}\{y \in \mathcal{Y}_{T^*}\}$.
- (4) If P_0 is corrupt, get a bit $\llbracket b \rrbracket_0$ from P_0 . Else sample $\llbracket b \rrbracket_0 \leftarrow \{0, 1\}$ and send it to P_0 .
- (5) Set $\llbracket b \rrbracket_1 := b \oplus \llbracket b \rrbracket_0$. Send $\llbracket b \rrbracket_1$ to P_1 .

Figure 10: Functionality $\mathcal{F}_{\text{membership}}$ for secure membership test.

THEOREM D.1. $\Pi_{\text{membership}}$ (Figure 5) securely realizes $\mathcal{F}_{\text{membership}}$ (Figure 10) in the $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_2}$ -hybrid model, with respect to a semi-honest adversary.

Proof. We first describe the simulator $\mathcal{S}$ for corrupt P_0 . In this case, $\mathcal{S}$ simply sends corrupt P_0 's input $\mathcal{Y}_{T^*}$ as input to $\mathcal{F}_{\text{membership}}$. Additionally, it internally executes $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_2}$ and receives the sender messages from corrupt P_0 . Finally, it sends the output z of corrupt P_0 to $\mathcal{F}_{\text{membership}}$ as the value $\llbracket b \rrbracket_0$. This simulated distribution is identical to the real world distribution.

We now describe the simulator $\mathcal{S}$ for corrupt P_1 . In this case, $\mathcal{S}$ first sends corrupt P_1 's input y as input to $\mathcal{F}_{\text{membership}}$. On receiving an output $\llbracket b \rrbracket_1$ from $\mathcal{F}_{\text{membership}}$, $\mathcal{S}$ internally executes $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_2}$ and receives P_1 's input y and responds with the value $\llbracket b \rrbracket_1$. This simulated distribution is identical to the real world distribution.

D.2 Secure Sampling from a Private Set

In Figure 11, we describe the ideal functionality for the secure membership test described in Section 4.1.

THEOREM D.2. Π_{rand} (Figure 3) securely realizes $\mathcal{F}_{\text{rand}}$ (Figure 11) in the $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ -hybrid model, with respect to passive corruption.

We first show a simulator for corrupt P_0 .

$\mathcal{S}_0$:

- (1) Send $\mathcal{Y}_{T^*}$ to $\mathcal{F}_{\text{rand}}$.
- (2) Invoke Π_{rand} with P_0 where P_1 is simulated as per the protocol description (with null input as P_1 does not have any input in the protocol). Let $z' \in \mathbb{Z}_T$ be the output of P_0 .
- (3) Send z' to $\mathcal{F}_{\text{rand}}$.

This simulated distribution is identical to the real world distribution. We now show a simulator for corrupt P_1 .

$\mathcal{S}_1$:

Functionality: $\mathcal{F}_{\text{rand}}$

Public parameters: Set $\mathcal{Y} = \{0, \dots, T-1\}$ and arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

- (1) Get $\mathcal{Y}_{T^*} \subseteq \mathcal{Y}_T$ as input from P_0 .
- (2) Sample $y \leftarrow \mathcal{Y}_{T^*}$.
- (3) If P_0 is corrupt, get $\llbracket y \rrbracket_0 \in \mathbb{Z}_T$ from P_0 . Else sample $\llbracket y \rrbracket_0 \leftarrow \mathbb{Z}_T$ and send it to P_0 .
- (4) Set $\llbracket y \rrbracket_1 := y - \llbracket y \rrbracket_0 \pmod{T}$. Send $\llbracket y \rrbracket_1$ to P_1 .

Figure 11: Functionality $\mathcal{F}_{\text{rand}}$ for secure sampling from a private set.

- (1) Receive a share $\llbracket y \rrbracket_1$ from $\mathcal{F}_{\text{rand}}$.
- (2) Internally execute $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ and receive OT sender strings from P_1 .
- (3) Internally execute $\mathcal{F}_{\text{OT}}^{(1), \mathbb{Z}_T}$ and receive OT receiver choice index from P_1 . Send $\llbracket y \rrbracket_1$ to P_1 .

We now define a series of hybrid experiments, with changes from one hybrid to another highlighted in blue.

- Hyb_0 : Real world
- Hyb_1 : Same as Hyb_0 except the following change: **In the second OT interaction, set the sender strings in the following way.** $\forall i \neq \llbracket v \rrbracket_1, u_i := 0$. For $i = \llbracket v \rrbracket_1, u_i := d_i - z' \pmod{N}$ where $d_i := (\llbracket v \rrbracket_0 + i \pmod{T})^{\text{th}}$ label.
- Hyb_2 : Same as Hyb_1 except the following change: **In the second OT interaction, set the sender strings in the following way.** $\forall i \neq \llbracket v \rrbracket_1, u_i := 0$. For $i = \llbracket v \rrbracket_1, u_i := d_i - z' \pmod{N}$ where $d_i \leftarrow \mathcal{Y}_{T^*}$.
- Hyb_3 : Same as Hyb_2 except the following change. **Get $\llbracket y \rrbracket_1$ from $\mathcal{F}_{\text{rand}}$.** In the second OT interaction, set the sender strings in the following way. $\forall i \neq \llbracket v \rrbracket_1, u_i := 0$. For $i = \llbracket v \rrbracket_1, u_i := \llbracket y \rrbracket_1$.
- Hyb_4 : Same as Hyb_3 except the following change: **In the first OT interaction, use 0 as the choice index.**

We note that the last hybrid Hyb_4 is trivially identical to the simulated distribution. We will now show that each pair of consecutive hybrids have an identical distribution.

- $\text{Hyb}_0 \equiv \text{Hyb}_1$: These hybrids are perfectly indistinguishable as the joint distribution of OT receiver output $u_{\llbracket v \rrbracket_1}$ from the second OT and output z' of honest P_0 is identical between Hyb_1 and Hyb_0 . In both cases, $(u_{\llbracket v \rrbracket_1}, z')$ form a random arithmetic sharing of $d_{\llbracket v \rrbracket_1}$, where $v = s_{T^*-1}$ and $d_{\llbracket v \rrbracket_1}$ is the v^{th} element in $\tilde{\mathcal{Y}}_{T^*}$.
- $\text{Hyb}_1 \equiv \text{Hyb}_2$: These hybrids are perfectly indistinguishable as the joint distribution of OT receiver output $u_{\llbracket v \rrbracket_1}$ from the second OT and output z' of honest P_0 is identical between Hyb_2 and Hyb_1 . In Hyb_1 , $(u_{\llbracket v \rrbracket_1}, z')$ form a random arithmetic sharing of $d_{\llbracket v \rrbracket_1}$, where $v = s_{T^*-1}$ and $d_{\llbracket v \rrbracket_1}$ is the v^{th} element in $\tilde{\mathcal{Y}}_{T^*}$. In Hyb_2 , $(u_{\llbracket v \rrbracket_1}, z')$ form a random arithmetic sharing of $d_{\llbracket v \rrbracket_1}$, where $d_{\llbracket v \rrbracket_1}$ is a random element from $\mathcal{Y}_{T^*}$. Note that this indistinguishability hold crucially

due to the fact that $\tilde{\mathcal{Y}}_{T^*}$ is a randomly permuted version of $\mathcal{Y}_{T^*}$.

- $\text{Hyb}_2 \equiv \text{Hyb}_3$: These hybrids are perfectly indistinguishable as the joint distribution of OT receiver output $u_{\llbracket v \rrbracket_1}$ from the second OT and output of honest P_0 (which is z' in Hyb_2 and $\llbracket y \rrbracket_0$ in Hyb_3) is identical between Hyb_3 and Hyb_2 . In Hyb_2 , $(u_{\llbracket v \rrbracket_1}, z')$ form a random arithmetic sharing of $d_{\llbracket v \rrbracket_1}$, where $d_{\llbracket v \rrbracket_1}$ is a random element from $\mathcal{Y}_{T^*}$. In Hyb_3 , $(u_{\llbracket v \rrbracket_1} = \llbracket y \rrbracket_1, \llbracket y \rrbracket_0)$ form a random arithmetic sharing of y , where y is a random element from $\mathcal{Y}_{T^*}$ sampled by $\mathcal{F}_{\text{rand}}$.
- $\text{Hyb}_3 \equiv \text{Hyb}_4$: These hybrids are perfectly indistinguishable as the joint distribution of the view of corrupt P_1 and the output of honest parties P_0 is unmodified.

D.3 Secure Bit Sampling with Private Bias

In Figure 12, we describe the ideal functionality for the secure biased bit sampling with private bias described in Section 4.1.

Functionality: $\mathcal{F}_{\text{biasBit}}$

Public parameter: Fixed point precision $f \in \mathbb{N}$.

- (1) Get $q_{\text{fix}} \in [0, 2^f)$ as input from P_0 .
- (2) Sample $b \leftarrow \text{Bernoulli}(\frac{q_{\text{fix}}}{2^f})$.
- (3) If P_0 is corrupt, get $\llbracket b \rrbracket_0$ from P_0 . Else sample $\llbracket b \rrbracket_0 \leftarrow \{0, 1\}$ and send it to P_0 .
- (4) Set $\llbracket b \rrbracket_1 := b \oplus \llbracket b \rrbracket_0$. Send $\llbracket b \rrbracket_1$ to P_1 .

Figure 12: Functionality $\mathcal{F}_{\text{biasBit}}$ for secure biased bit sampling with private bias.

THEOREM D.3. Π_{biasBit} securely realizes $\mathcal{F}_{\text{biasBit}}$ in the $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ hybrid model, with respect to passive corruption.

Proof. We first describe the simulator $\mathcal{S}$ for corrupt P_0 . In this case, $\mathcal{S}$ sends P_0 's input q_{fix} to $\mathcal{F}_{\text{biasBit}}$. Then, it internally executes $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ and receives the sender messages from corrupt P_0 . Finally, it sends the output z of corrupt P_0 to $\mathcal{F}_{\text{biasBit}}$ as the value $\llbracket b \rrbracket_0$. This simulated distribution is identical to the real world distribution.

We now describe the simulator for corrupt P_1 . In this case, $\mathcal{S}$ first receives $\llbracket b \rrbracket_1$ from $\mathcal{F}_{\text{biasBit}}$. Then $\mathcal{S}$ internally executes $\mathcal{F}_{\text{OT}}^{(2^f), \mathbb{Z}_2}$ where it receives corrupt P_1 's choice index $r \in [0, 2^f)$ and responds with the value $\llbracket b \rrbracket_1$. This simulated distribution is identical to the real world distribution.

D.4 Secure Multiplexer

4 choose 1 mux. In Figure 13, we describe the ideal functionality for 4-choose-1 secure multiplexer introduced in Section 4.1.4. Figure 14 describes the corresponding protocol $\Pi_{\text{mux}}^{(4)}$ for realizing the functionality $\mathcal{F}_{\text{mux}}^{(4)}$. At a high-level, $\Pi_{\text{mux}}^{(4)}$ performs two calls to $\mathcal{F}_{\text{OT}}^{(4), \mathbb{Z}_T}$ where the first (resp. second) OT is used to generate a sharing of $\llbracket x^{c,d} \rrbracket_0$ (resp. $\llbracket x^{c,d} \rrbracket_1$) across both parties. Once parties have

a sharing of $\llbracket x^{c,d} \rrbracket_0$ and $\llbracket x^{c,d} \rrbracket_1$, they can add the shares locally (due to linearity of the sharing scheme) to derive a sharing of $x^{c,d}$. We now explain the working of first OT; the second OT works in a similar way but with the sender and receiver roles reversed. In the first OT, P_0 acts as the OT sender and sets up the sender strings in such a way that when P_1 , acting as the OT receiver, uses $(\langle c \rangle_1, \langle d \rangle_1)$ as the choice index, then it should retrieve (a masked version of) $\llbracket x^{c,d} \rrbracket_0$. P_0 can set up the OT sender strings in this desired fashion simply by enumerating over possible values of $(\langle c \rangle_1, \langle d \rangle_1)$ (there are 4 possible values).

Functionality: $\mathcal{F}_{\text{mux}}^{(4)}$

Public parameter: Arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

- (1) Get $\llbracket x^{0,0} \rrbracket_0, \llbracket x^{0,1} \rrbracket_0, \llbracket x^{1,0} \rrbracket_0, \llbracket x^{1,1} \rrbracket_0, \langle c \rangle_0, \langle d \rangle_0$ from P_0
- (2) Get $\llbracket x^{0,0} \rrbracket_1, \llbracket x^{0,1} \rrbracket_1, \llbracket x^{1,0} \rrbracket_1, \llbracket x^{1,1} \rrbracket_1, \langle c \rangle_1, \langle d \rangle_1$ from P_1
- (3) For all $i, j \in \{0, 1\} \times \{0, 1\}$, reconstruct $x^{i,j} := \llbracket x^{i,j} \rrbracket_0 + \llbracket x^{i,j} \rrbracket_1 \pmod{T}$.
- (4) Compute $c := \langle c \rangle_0 \oplus \langle c \rangle_1$ and $d := \langle d \rangle_0 \oplus \langle d \rangle_1$.
- (5) Set $y := x^{c,d}$.
- (6) Sample $\llbracket y \rrbracket_0 \leftarrow \mathbb{Z}_T$.
- (7) Set $\llbracket y \rrbracket_1 := y - \llbracket y \rrbracket_0 \pmod{\mathbb{Z}_T}$.
- (8) Send $\llbracket y \rrbracket_0$ to P_0 and $\llbracket y \rrbracket_1$ to P_1 .

Figure 13: Functionality $\mathcal{F}_{\text{mux}}^{(4)}$ for 4-choose-1 secure multiplexer.

THEOREM D.4. $\Pi_{\text{mux}}^{(4)}$ securely realizes $\mathcal{F}_{\text{mux}}^{(4)}$ in the $\mathcal{F}_{\text{OT}}^{(4), \mathbb{Z}_T}$ -hybrid model, with respect to passive corruption.

2 choose 1 mux. In Figure 15, we describe the ideal functionality for 2-choose-1 secure multiplexer. Figure 16 describes the corresponding protocol $\Pi_{\text{mux}}^{(2)}$ for realizing the functionality $\mathcal{F}_{\text{mux}}^{(2)}$.

THEOREM D.5. $\Pi_{\text{mux}}^{(2)}$ securely realizes $\mathcal{F}_{\text{mux}}^{(2)}$ in the $\mathcal{F}_{\text{OT}}^{(2), \mathbb{Z}_T}$ -hybrid model, with respect to passive corruption.

D.5 Secure RRWithPrior

The security of $\Pi_{\text{RRWithPrior}}$ is formally captured using the following theorem.

THEOREM D.6. $\Pi_{\text{RRWithPrior}}$ (Figure 6) securely realizes $\mathcal{F}_{\text{RRWithPrior}}$ (Figure 2) in the $\mathcal{F}_{\text{membership}}, \mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{mux}}^{(1)}, \mathcal{F}_{\text{rand}}$ hybrid model, with respect to passive corruption.

Proof. We first describe the simulator $\mathcal{S}$ for corrupt P_0 . Note that since the honest P_1 does not have any output, we just need to ensure that the view of corrupted P_0 in the simulated distribution is indistinguishable from its view in the real protocol. $\mathcal{S}$ starts by sending P_0 's input $\tilde{p}$ to $\mathcal{F}_{\text{RRWithPrior}}$ and receives $\tilde{y}$ from $\mathcal{F}_{\text{RRWithPrior}}$. It then internally executes $\mathcal{F}_{\text{biasBit}}, \mathcal{F}_{\text{membership}}, \mathcal{F}_{\text{rand}}$ where it receives $(q_{\text{fix}}, \llbracket b_1 \rrbracket_0), (\mathcal{Y}^*, \llbracket b_2 \rrbracket_0), (\mathcal{Y}^*, \llbracket z \rrbracket_0)$ respectively from P_0 . It then internally executes $\mathcal{F}_{\text{mux}}^{(4)}$ where it receives selection $(\llbracket b_1 \rrbracket_0, \llbracket b_2 \rrbracket_0)$ and value shares $(\llbracket z \rrbracket_0, \llbracket z \rrbracket_1, \llbracket y \rrbracket_0)$ from P_0 .

Protocol: $\Pi_{\text{mux}}^{(4)}$

Public input: Arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

Private input: $\llbracket x^{0,0} \rrbracket, \llbracket x^{0,1} \rrbracket, \llbracket x^{1,0} \rrbracket, \llbracket x^{1,1} \rrbracket, \langle c \rangle, \langle d \rangle$.

Output: $\llbracket y \rrbracket$ where $y = x^{c,d}$.

- (1) P_0 samples $r_0 \leftarrow \mathbb{Z}_T$ and P_1 samples $r_1 \leftarrow \mathbb{Z}_T$.

- (2) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(4), \mathbb{Z}_T}$:

- P_0 acts as the OT sender using strings:

$$\begin{aligned} (s_0, s_1, s_2, s_3) &= (\llbracket x^{\langle c \rangle_0, \langle d \rangle_0} \rrbracket_0 - r_0, \\ &\quad \llbracket x^{\langle c \rangle_0, \neg \langle d \rangle_0} \rrbracket_0 - r_0, \\ &\quad \llbracket x^{\neg \langle c \rangle_0, \langle d \rangle_0} \rrbracket_0 - r_0, \\ &\quad \llbracket x^{\neg \langle c \rangle_0, \neg \langle d \rangle_0} \rrbracket_0 - r_0) \end{aligned}$$

- P_1 acts as the OT receiver using $\langle c \rangle_1, \langle d \rangle_1$ as the choice bit.

- P_1 retrieves $y_1 := s_{2\langle c \rangle_1 + \langle d \rangle_1}$.

- (3) P_1 and P_0 invoke a $\mathcal{F}_{\text{OT}}^{(4), \mathbb{Z}_T}$:

- P_1 acts as the OT sender using strings:

$$\begin{aligned} (s'_0, s'_1, s'_2, s'_3) &= (\llbracket x^{\langle c \rangle_1, \langle d \rangle_1} \rrbracket_1 - r_1, \\ &\quad \llbracket x^{\langle c \rangle_1, \neg \langle d \rangle_1} \rrbracket_1 - r_1, \\ &\quad \llbracket x^{\neg \langle c \rangle_1, \langle d \rangle_1} \rrbracket_1 - r_1, \\ &\quad \llbracket x^{\neg \langle c \rangle_1, \neg \langle d \rangle_1} \rrbracket_1 - r_1) \end{aligned}$$

- P_0 acts as the OT receiver using $\langle c \rangle_0, \langle d \rangle_0$ as the choice bit.

- P_0 retrieves $y_0 := s'_{2\langle c \rangle_0 + \langle d \rangle_0}$.

- (4) For $b \in \{0, 1\}$, P_b outputs $y_b + r_b$.

Figure 14: Protocol for 4-choose-1 secure multiplexer.

Functionality: $\mathcal{F}_{\text{mux}}^{(2)}$

Public parameter: Arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

- (1) Get $\llbracket x^0 \rrbracket_0, \llbracket x^1 \rrbracket_0, \langle c \rangle_0$ from P_0
- (2) Get $\llbracket x^0 \rrbracket_1, \llbracket x^1 \rrbracket_1, \langle c \rangle_1$ from P_1
- (3) For all $i \in \{0, 1\}$, reconstruct $x^i := \llbracket x^i \rrbracket_0 + \llbracket x^i \rrbracket_1 \pmod{T}$.
- (4) Compute $c := \langle c \rangle_0 \oplus \langle c \rangle_1$.
- (5) Set $y := x^c$.
- (6) Sample $\llbracket y \rrbracket_0 \leftarrow \mathbb{Z}_T$.
- (7) Set $\llbracket y \rrbracket_1 := y - \llbracket y \rrbracket_0 \pmod{\mathbb{Z}_T}$.
- (8) Send $\llbracket y \rrbracket_0$ to P_0 and $\llbracket y \rrbracket_1$ to P_1 .

Figure 15: Functionality for 2-choose-1 secure multiplexer.

Finally, it sends $\llbracket y \rrbracket_1 := \tilde{y} - \llbracket y \rrbracket_0 \pmod{T}$ to P_0 . This simulated view is identical to the real world distribution.

We now describe the simulator $\mathcal{S}$ for corrupt P_1 . $\mathcal{S}$ sends P_1 's input y to $\mathcal{F}_{\text{RRWithPrior}}$. Then it internally executes $\mathcal{F}_{\text{biasBit}}$ and $\mathcal{F}_{\text{rand}}$ where it receives nothing from P_1 and sends random $\llbracket b_1 \rrbracket_1$ and $\llbracket z \rrbracket_1$ respectively to P_1 . $\mathcal{S}$ internally executes $\mathcal{F}_{\text{membership}}$ where it

Protocol: $\Pi_{\text{mux}}^{(2)}$

Public input: Arithmetic sharing domain $\mathbb{Z}_T$, where $T \in \mathbb{N}$.

Private input: $\llbracket x^0 \rrbracket, \llbracket x^1 \rrbracket, \langle c \rangle$.

Output: $\llbracket y \rrbracket$ where $y = x^c$.

- (1) P_0 samples $r_0 \leftarrow \mathbb{Z}_T$ and P_1 samples $r_1 \leftarrow \mathbb{Z}_T$.
- (2) P_0 and P_1 invoke a $\mathcal{F}_{\text{OT}}^{(2), \mathbb{Z}_T}$:
 - P_0 acts as the OT sender using strings $(s_0, s_1) = (\llbracket x^{(c)_0} \rrbracket_0 - r_0, \llbracket x^{-(c)_0} \rrbracket_0 - r_0)$
 - P_1 acts as the OT receiver using $\langle c \rangle_1$ as the choice bit.
 - P_1 retrieves $y_1 := s_{\langle c \rangle_1}$.
- (3) P_1 and P_0 invoke a $\mathcal{F}_{\text{OT}}^{(2), \mathbb{Z}_T}$:
 - P_1 acts as the OT sender using strings $(s'_0, s'_1) = (\llbracket x^{(c)_1} \rrbracket_1 - r_1, \llbracket x^{-(c)_1} \rrbracket_1 - r_1)$
 - P_0 acts as the OT receiver using $\langle c \rangle_0$ as the choice bit.
 - P_0 retrieves $y_0 := s'_{\langle c \rangle_0}$.

- (4) For $b \in \{0, 1\}$, P_b outputs $y_b + r_b \pmod{T}$.

Figure 16: Protocol for 2-choose-1 secure multiplexer.

receives y as input from P_1 and sends a random $\llbracket b_2 \rrbracket_1$ to P_1 . $\mathcal{S}$ internally executes $\mathcal{F}_{\text{mux}}^{(4)}$ where it receives selection $(\llbracket b_1 \rrbracket_1, \llbracket b_2 \rrbracket_1)$ and value shares $(\llbracket z \rrbracket_1, \llbracket z \rrbracket_1, \llbracket z \rrbracket_1, \llbracket y \rrbracket_1)$ from P_1 . Finally, it receives

$\llbracket \tilde{y} \rrbracket_1$ from P_1 . This simulated view is identical to the real world distribution.

D.6 Secure Interval Finding

In Figure 17, we present the ideal functionality for the secure interval finding described in Section C.

Functionality: $\mathcal{F}_{\text{interval}}$

Public parameter: Set $S = [0, R) \subset \mathbb{N}$, where $R \in \mathbb{N}$

- (1) Get a list of k buckets/partitions $\mathcal{P} = \{[c_0, c_1), \dots, [c_{k-1}, c_k)\}$ where each $c_i \in [0, R)$, $c_0 = 0$ and $c_k = R$ as input from P_0 .
- (2) Get a value $d \in [0, R)$ as input from P_1 .
- (3) Let $z' \in [0, k-1]$ indicate the partition index s.t. $c_{z'} \leq d < c_{z'+1}$.
- (4) If P_0 is corrupt, get $\llbracket z' \rrbracket_0 \in \mathbb{Z}_R$ as input from $\mathcal{S}$. Else sample $\llbracket z' \rrbracket_0 \leftarrow \mathbb{Z}_R$ and send it to P_0 .
- (5) Compute $\llbracket z' \rrbracket_1 := z' - \llbracket z' \rrbracket_0 \pmod{R}$. Send $\llbracket z' \rrbracket_1$ to P_1 .

Figure 17: Functionality for secure interval finding.

THEOREM D.7. Π_{interval} securely realizes $\mathcal{F}_{\text{interval}}$ in the $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ -hybrid model, with respect to passive corruption.

Proof. The proof follows directly having the simulator emulate $\mathcal{F}_{\text{OT}}^{(R), \mathbb{Z}_R}$ in the ideal world.